

LOOPS BENCH: From Harness Engineering to Loop Engineering in Coding Agent Evaluation

Han Li^{1,†,*} Zheming Fang^{2,*} Rili Feng^{2,*} Yingqi Zhao⁴ Jiaheng Liu² Pengfei Gao^{1,†}
He Ye³ Dayi Lin¹ Qingwei Lin¹ Saravan Rajmohan¹ Dongmei Zhang¹

¹Microsoft ²Nanjing University ³University College London ⁴Shanghai Jiao Tong University

Project page: <https://loopsbench.ai/>

*Equal contribution. †Corresponding author. ‡Work done during an internship at Microsoft.

Abstract

Coding agent infrastructure is shifting from harness engineering toward loop engineering as coding agents are deployed for sustained long horizon software development. Existing benchmarks often center on localized tasks or end state outcomes, offering limited insight into sustained execution. We introduce LOOPS BENCH, a long horizon benchmark for loop engineering in coding agent evaluation. Each task is a dependency DAG over separately testable development units with source evidenced prerequisite edges. LOOPS BENCH comprises 112 tasks from authentic sources spanning 8 programming languages and 9 domains. Its flow aware runtime releases tests along the ready frontier and retains completed nodes as regression obligations. We evaluate frontier coding agents paired with widely used loop implementations. The strongest configuration, Opus-4.7 with Claude Code and outer continuation, resolves 25.00% of tasks. Recorded plans recover only part of the source recovered prerequisite DAG, and regression events remain visible across the evaluated loop profiles. We open source the benchmark data and code, including all tasks, more than 5,300 development units, and executable tests, at [microsoft/Loopsbench](https://microsoft.com/Loopsbench).

1 Introduction

Current coding agent systems increasingly expose loop mechanisms for sustained software work, e.g., Codex goal mode, Claude Code goal mode, and Claude Code dynamic workflows OpenAI (2026a); Anthropic (2026a,c,b). These mechanisms do not replace the harness. They add a higher level control surface over it, so objectives, progress criteria, and work distribution can persist across extended execution. The core challenge therefore moves from harness engineering alone to loop engineering over the harness. In long-horizon coding, the loop must govern execution across task structure, state continuity, and regression pressure as dependent work accumulates Carlini (2026).

Limitations of Existing Benchmarks. Existing benchmarks reflect this regime only partially. SWE-bench Jimenez et al. (2023a) and its variants broaden repository level issue resolution along freshness, language coverage, and patch scale Zan et al. (2025); Yang et al. (2024c); Liu et al. (2025). Feature level benchmarks Zhou et al. (2026); Chen et al. (2025); Thai et al. (2025) further expand the requested change. Their task abstraction nevertheless remains largely terminal: agents receive self contained issues or flat specifications and are judged by final task success. This design measures issue resolution ability but does not reveal whether an agent preserves intermediate obligations, avoids regressions, or follows a viable order through dependent subproblems. Diagnostic long horizon evaluation should expose intermediate development units, track accumulated obligations, and make execution order observable.

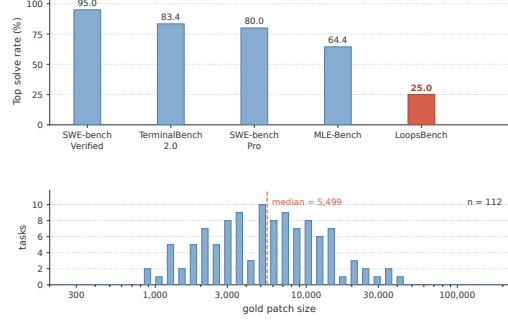
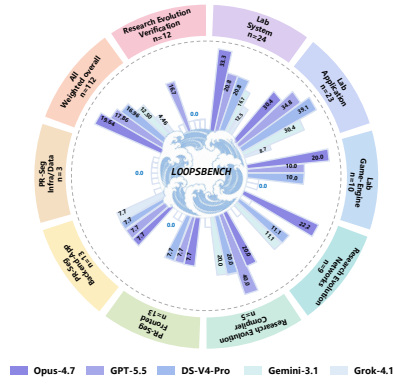


Figure 1: Summary statistics and results for LOOPSBENCH.

Table 1: Comparison of LOOPSBENCH with existing coding benchmarks.

Benchmark	Evidence	Lng	Task Form	Units	DAG	Pch	Tst	Time
LOOPSBENCH	loop trace metrics	8	dependency DAG	✓	✓	37,296	74	6.6m
<i>SWE family</i>								
SWE-bench	end state F2P	1	issue	✗	✗	33	9	24.6d
Multi-SWE-bench	end state F2P	7	issue	✗	✗	107	21	18.5d
SWE-bench Pro	end state F2P	11	issue or feature	✗	✗	462	38	1.7m
FeatureBench	end state F2P	1	feature	partial	✗	1,256	38	19.5d
SWE-EVO	end state F2P	1	software evolution	partial	✗	610	31	7.1d
<i>Long horizon family</i>								
MLE-Bench	end state F2P	1	repo generation	✗	✗	514	1	3.0m
NL2Repo-Bench	end state F2P	1	repo generation	partial	✗	1,623	247	3.4y
ProgramBench	end state F2P	1	repo generation	partial	✗	2,104	52	12.4d
RepoZero	end state F2P	1	repo generation	partial	✗	3,287	68	21.7d

LOOPSBENCH as a Benchmark for Loop Engineering Evaluation. LOOPSBENCH (Figure 1) addresses these requirements by representing each task as a dependency DAG whose nodes are source grounded development units and whose edges encode prerequisite relations. The graph makes intermediate units separately testable and provides a source recovered development order as a descriptive reference for sequencing analysis, not as a claim of optimality. The flow aware evaluation runtime releases ready frontier tests, keeps completed nodes active as regression obligations, and records a loop trace while leaving execution order open. LOOPSBENCH contains 112 dependency-structured tasks from three authentic, real-world sources, with 29 PR Sequences, 57 Course Labs, and 26 Research Evolutions spanning 9 domains and 8 programming languages. The benchmark has a median dependency depth of 6 and more than 5,300 development units. Table 1 distinguishes benchmarks that expose separately testable intermediate units from those that only specify terminal tasks, and further separates unit exposure from explicit prerequisite DAGs. This comparison situates LOOPSBENCH as, to our knowledge, the first benchmark for loop engineering evaluation, pairing explicit unit dependency DAGs with a flow-aware harness and loop trace diagnostics.

Takeaways. Three findings emerge. First, the strongest model and loop configuration resolves only 25.00% of LOOPSBENCH. Second, evaluated loops omit prerequisite relations, produce longer patches, and author sparse tests. Third, context renewal differs across loop implementations, while regression events remain visible in every profile, identifying routing and state tracking as central limitations.

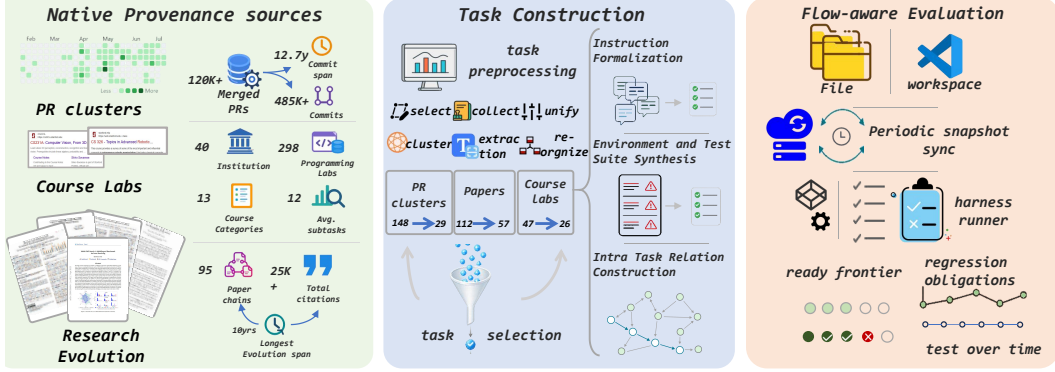


Figure 2: Overview of the LOOPS BENCH construction and evaluation pipeline.

Contributions. LOOPS BENCH provides a graph structured evaluation contract, a scalable task construction pipeline, and 112 source grounded long horizon tasks. Its trace based analysis separates model and loop effects while measuring planning, implementation, testing, routing, and state retention.

2 LOOPS BENCH as a Pipeline for Loop Engineering Evaluation in Long Horizon Coding

Figure 2 summarizes the pipeline components, while the numbered stages below specify their construction order. LOOPS BENCH constructs loop engineering evaluation instances in which dependency structure controls test release, completed work remains live as regression obligations, and each run records loop trace metrics for planning, routing, verification, state continuity, and obligation retention. (1) **Task Collection** gathers source artifacts from three authentic real world sources. (2) **Task Preprocessing** normalizes the source artifacts into atomic candidate tasks. (3) **Task Selection** filters candidates by two source agnostic thresholds. (4) **Development Unit Definition and Intra Task Relation Recovery** defines the units and recovers the dependency DAG that governs ready frontier release. (5) **Task Instrumentation** then attaches the public instruction, executable environment, and per unit test obligations using the recovered graph.

2.1 Task Collection

We collect source artifacts from three authentic real world sources spanning a broad spectrum of coding domains, task shapes, and dependency depths. This source mix varies routing burden and obligation retention pressure while keeping all tasks under the same loop engineering evaluation contract. (1) **Course Labs.** We collect 112 university programming projects from the official course homepages of 30 universities and from public GitHub mirrors of their assignment releases, retrieving each as its complete native bundle of handout, skeleton code, and instructor side reference tests. (2) **PR Sequences.** We pull the full commit history and merged pull request stream of 56 actively maintained GitHub repositories, all under continuous community development with high star counts and long maintenance windows. (3) **Research Evolutions.** We gather 212 papers from 48 venues together with the arXiv tex of their cited prior work and forward citers, restricted to papers that are simultaneously highly cited and accompanied by a fully open source official implementation under an OSI approved license. A reference filter retains the 95 papers connected by load bearing inheritance edges. The three sources jointly anchor the benchmark in authentic developer, educator, and researcher artifacts and yield complementary long horizon structures for evaluating coding agent loops. Per source coverage appears in Appendix B.

2.2 Task Preprocessing

We normalize the collected artifacts into atomic candidate tasks before selection so later stages can represent heterogeneous sources through a uniform evaluation contract.

Course Labs. ❶ unify handouts written in mixed languages into a single English version; ❷ reorganize scattered skeleton files under one task root. Each project becomes one candidate task, yielding 112 candidates across the collected projects.

PR Sequences. ❶ clone each repository with full git history and snapshot the first commit as the base codebase; ❷ collect its merged pull requests and extract per pull request diffs split by role (e.g., test, source, auxiliary); ❸ run a Claude Code based extraction step Anthropic (2025d) that performs repository level exploration and emits aggregated segments validated against a fixed acceptance standard. The procedure yields 148 candidate tasks across the 56 repositories.

Research Evolutions. ❶ select a high impact early seed paper in the collected venue set and collect the arXiv tex of the seed together with its cited prior work and forward citers; ❷ run a Claude Code based classification step that labels each reference as a shallow related work mention or a load bearing inheritance edge. ❸ absorb every neighbor connected by a load bearing edge into the cluster and repeat the classification on the newly added papers until the closure admits no further additions. Each closed cluster becomes one candidate task, yielding 47 candidates across the 95 retained papers; 26 survive Selection. Across all three sources, heterogeneous native test runners are wrapped behind a uniform task entry point, ensuring that every evaluated loop observes the same runtime contract while the source specific dependency evidence is preserved. Appendix D presents one materialized LOOPS BENCH task, from candidate recovery to the loop trace metrics recorded during evaluation.

2.3 Task Selection

Candidate tasks pass two source agnostic thresholds that retain workloads of sufficient scale for routing burden, state continuity, and regression obligation retention to become observable. **Temporal Span** is at least 2.5 months, measured as the handout declared project duration for Course Labs, the merge time interval between the first and last commit of the region for PR Sequences, and the publication date span between the earliest and latest paper in the evolution for Research Evolutions. **Solution Scale** is at least 1,200 in the source specific scale. Each source has its own natural patch length and base codebase size. Per source values are reported in Appendix B. After filtering, 112 tasks remain as the released loop engineering evaluation suite.

2.4 Intra Task Relation Recovery

A materialized development unit is a source grounded, separately testable acceptance unit $u = (r_u, s_u, p_u, \Delta_u, T_u)$, where r_u is its requirement, s_u its file or symbol scope, p_u its prerequisite set, Δ_u its reference patch contribution, and T_u its standard tests. Unit boundaries follow source provenance: merged PRs or evidence supported PR segments for PR Sequences, released modules or milestones for Course Labs, and load bearing methodological increments for Research Evolutions. Changes that cannot be separated without breaking the acceptance contract are assigned to the nearest prerequisite unit or merged rather than split artificially.

We formalize each task as a directed acyclic graph over its development units, allowing loop engineering evaluation to observe how a loop routes work through unit level prerequisites. The fixed DAG is an evaluation contract rather than a claim that real development is always monotonic or interface stable. An evaluated loop may revise earlier implementations provided that the completed predecessor obligations remain satisfied; tasks that intentionally replace earlier requirements through exploratory redesign fall outside this abstraction. The four admitted patterns are *sequential PR chains* along the merged commit history, *structural module reuse* when v edits a file or symbol that u introduced, *functional producer consumer API edges* when v calls or imports an authoritative definition created by u , and *compositional layering* when v extends a subclass, schema, or interface declared by u .

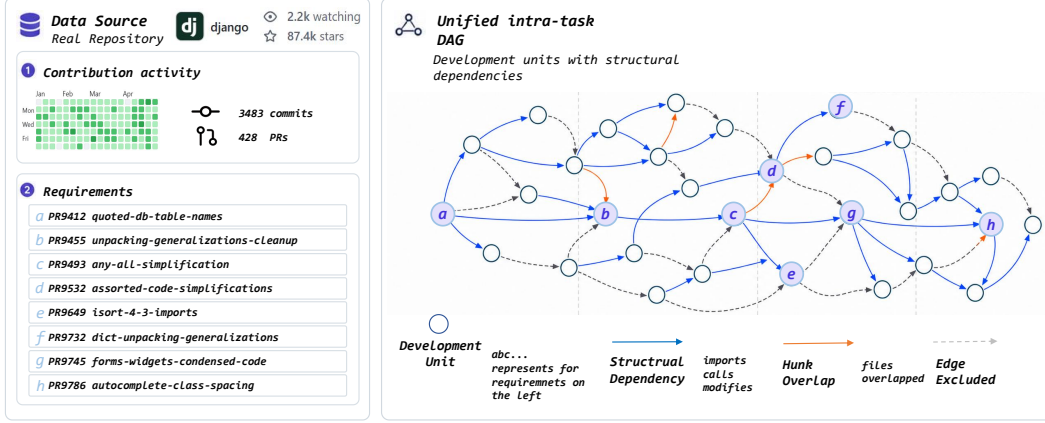


Figure 3: Four prerequisite patterns in a materialized task.

An edge is admitted only under unambiguous evidence on the gold artifacts. The first kind of evidence is hunk overlap on the same file. The second kind of evidence is symbol level dependence through an import, call, or subclass of an authoritative definition introduced by u . Spurious chains from hot monoliths, generated artifacts, and lockfiles are excluded by a denylist applied before edge admission. A file touched by most PRs in a repository therefore contributes no edges, and a regenerated lockfile never anchors a successor. The full admission rules appear in Appendix C. On G , the *ready frontier* at checkpoint t is

$$R_t = \{v \in V \setminus C_t : \forall (u, v) \in E, u \in C_t\},$$

where C_t collects nodes whose tests all pass at t . For loop engineering evaluation, edges encode how local requirements become persistent obligations that remain active as later units are attempted. The tests for each unit are evaluated in a workspace where predecessor tests remain enforced, measuring progress on later units together with preservation of earlier obligations.

2.5 Task Instrumentation

We turn each selected candidate into a self contained evaluation instance whose evaluation contract contains a formalized task instruction, a reproducible environment, and discriminative tests. The gold solution stays outside the evaluated loop context and is used only to validate executable obligations and compute reference statistics before evaluation begins.

❶ Instruction Formalization. We treat the gold diff against the base codebase as ground truth and recover the developer’s acceptance intent from it, since PR titles, course handouts, and paper method sections rarely specify the full executable contract on their own. A Claude Code based recovery step Anthropic (2025d) backed by Claude Opus 4.7 Anthropic (2026d) produces a single acceptance instruction while keeping the implementation path hidden. The recovery step has two roles in the evaluation contract. It removes evidence of the implementation path, including concrete edit locations and gold diff structure. It retains the public acceptance contract that the test suite binds to, including public signatures and expected output formats. A symbol enters the instruction when a test assertion depends on naming it. The same recovery step strips repository names, course identifiers, paper titles, and author attributions to reduce prompt level source recognition.

❷ Environment and Test Suite Synthesis. A Claude Code based environment synthesis step traverses the source frame from upstream preprocessing, collects runtime dependencies, and writes a Dockerfile for the environment, with a companion `docker-compose.yaml` when the task spans multiple services. After development units and their dependency DAG have been recovered, the materialization step traverses the DAG in topological order so each unit test is authored against the prerequisite state used for ready frontier release. At each unit u , the materialization pipeline drafts missing tests on top of the current environment, applies u ’s gold patch on the recorded base, and runs the new tests under a three trial fail to pass check that admits $T(u)$ when the full gold solution flips every test to pass (*solvability*), the empty workspace flips none (*non triviality*), and the gold solution restricted to u ’s DAG ancestors still leaves at least one test failing (*discriminateness* with respect to u ’s own contribution). A test with no execution result is attributed to the environment and triggers a Dockerfile revision, while a test that executes yet remains nonpassing under the gold patch is

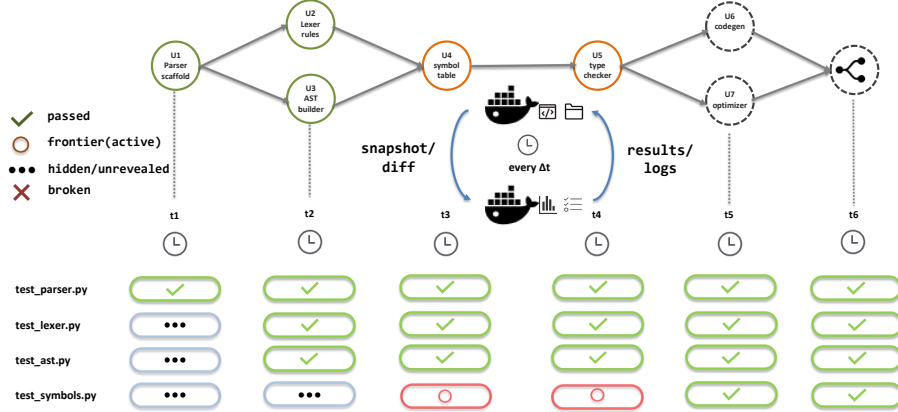


Figure 4: Flow aware evaluation runtime.

attributed to the assertion and rewritten in place. After every unit has been visited, a closure pass replays the full gold patch against the accumulated suite on the recorded base. This grounds loop traces in executable obligations and excludes textual milestones from the scoring contract. Units with unrepaired environment or suite issues after the computational budget are discarded.

2.6 Evaluation Protocol

As illustrated in Figure 4, the evaluated loop receives the task instruction and per unit requirement bundles as plain files, while gold solutions and the ready unit identity remain withheld. Checkpoint outcomes and the active obligation state are retained for evaluator side scoring; the evaluated loop can inspect the attached tests, use project native tests, and author tests of its own. Appendix C gives the complete visibility contract in Table 8. The evaluation runtime records the visitation sequence in the loop trace and compares the prerequisite structure in the recorded plan with the source recovered DAG. It accepts any valid topological execution order rather than requiring reproduction of the historical total order.

Topologically Gated Test Release. The evaluation runtime releases tests layer by layer along the dependency DAG, with every multi predecessor node acting as a gate. Here, release and sealing refer only to evaluator side activation for scoring, not disclosure of the attached test files or bindings. A gate node v joins the ready frontier after all predecessors lie in C_t , and the tests of its descendants stay sealed until $T(v)$ flips from fail to pass. The gate affects scoring rather than editing permission. The evaluated loop remains free to edit across layers, and the recorded loop trace reports whether its work satisfies the dependency frontier without being told which unit is currently ready. Once a unit clears the gate, its tests and the tests of its predecessors are kept enforced as regression tests on every later layer, scoring subsequent edits against both new work and completed obligations. Regression Rate therefore measures preservation of previously satisfied behavior without staged evaluator feedback about checkpoint outcomes or the active obligation state.

Dual Container Snapshot Pipeline. We separate edit execution from test adjudication across two containers, allowing the evaluated loop to continue editing while the evaluation runtime independently samples workspace state and updates released obligations. Container A hosts the working tree and is the sole write target for the evaluated loop. Container B contains the full test suite and the reference environment, and reads from A through snapshots managed by the evaluation runtime. On a fixed time cadence, the evaluation runtime samples A and diffs the new snapshot against the queue head. Empty diffs are dropped, while non empty diffs are appended to the queue and shipped into B for the currently released layer to run against. Snapshots form a queue of monotonic edit states, allowing LOOPS BENCH to compute loop trace metrics at real change points as well as at the final state.

3 Experiments

We organize the experiments around three questions that treat coding agent evaluation as loop engineering evaluation. All experiments use the full 112 task release of LOOPS BENCH.

Table 2: RQ1 performance across models and loop implementations. Per task billed tokens are reported in Appendix J.

Model	RR↑		TPR↑		Depth↑		RR↑		TPR↑		Depth↑	
	w/o	w/	w/o	w/			w/o	w/	w/o	w/		
GPT (Codex)							Model (fixed loop: Claude Code)					
GPT-5.5	14.29%	21.43%	39.62%	50.84%	0.53		Opus-4.7	16.96%	25.00%	41.18%	53.05%	0.61
GPT-5.2	9.82%	13.39%	33.91%	43.90%	0.36		GPT-5.5	13.39%	20.54%	38.04%	49.30%	0.47
GPT-5	5.36%	8.04%	28.45%	36.86%	0.22		GLM-5.1	13.39%	18.75%	34.55%	45.49%	0.44
GPT-4o	1.79%	3.57%	21.85%	28.30%	0.09		DeepSeek-V4P	11.61%	18.75%	36.07%	47.34%	0.46
Claude (Claude Code)							Gemini-3.1-Pro	8.93%	14.29%	32.86%	43.59%	0.31
Opus-4.7	16.96%	25.00%	41.18%	53.05%	0.61		Qwen3.6-Plus	6.25%	9.82%	36.43%	47.50%	0.34
Opus-4.5	12.50%	17.86%	36.97%	48.21%	0.46		Kimi-2.6	3.57%	6.25%	21.27%	28.33%	0.14
Sonnet-4.5	8.93%	12.50%	32.85%	42.58%	0.34		Grok-4.1-FR	2.68%	4.46%	23.32%	30.97%	0.15
Sonnet-4	4.46%	7.14%	27.43%	35.54%	0.18		Loop (fixed model: gpt-5.4)					
Qwen (Qwen Code)							Codex	13.39%	18.75%	37.84%	49.06%	0.49
Qwen3.6-Plus	6.25%	9.82%	37.97%	45.10%	0.31		Claude Code	12.50%	17.86%	36.97%	48.21%	0.42
Qwen3.5-Plus	5.36%	8.04%	31.75%	41.17%	0.19		GitHub Copilot	10.71%	15.18%	34.21%	45.46%	0.38
Qwen3-Max	0.89%	1.79%	17.45%	23.31%	0.07		OpenHands	6.25%	9.82%	29.21%	39.30%	0.20
Qwen2.5-72B	0.00%	0.00%	10.61%	13.88%	0.03		SWE-agent	5.36%	8.93%	28.07%	38.11%	0.16
							mini-swe-agent	4.46%	7.14%	25.96%	35.20%	0.13

3.1 RQ1: How Far Do Coding Agent Loops Sustain Progress?

Table 2 reports RQ1 results along three dimensions that separate model choice, loop implementation, and residual continuation. The table covers within family model scaling under the corresponding vendor loop, model sweeps under a fixed loop implementation, and loop implementation sweeps under a fixed model. Across all three dimensions, every cell remains well below full resolution of LOOPS BENCH, suggesting that long horizon limitations are not explained by model choice or loop implementation in isolation. The strongest model and loop configuration resolves 25.00% of tasks. Without the outer evaluation loop, a single evaluated execution segment usually reaches a shallow dependency prefix. These pairings align frontier models with the vendor infrastructure under which they are typically deployed, yet long horizon loop execution remains well below complete resolution. The loop implementation sweep isolates how the surrounding execution loop changes progress under the same model. Without external continuation, evaluated loop implementations cover a shallow prefix under their own state retention and work routing policies. The recorded loop traces also capture partial progress patterns, including unresolved units with empty patches and skipped obligations that prioritize lower burden units. External continuation reduces early stalls and increases partial progress, while residual routing and state retention remain central loop engineering factors. In the model dimension, Resolve Rate plateaus even when the outer evaluation loop attempts every unit, because later units inherit earlier open obligations. Even under tool use and execution oriented training, the evaluated model and loop configurations often decompose the task into local issues without consistently maintaining a persistent global plan. Appendix I reports pass@K to check that within family monotonicity is not driven by a single trial. In Table 2, RR denotes Resolve Rate, TPR denotes Test Pass Rate, and Depth is normalized dependency depth. Mean billed tokens per task appear in Appendix J.

RQ1: Sustained long horizon loop progress remains limited at frontier scale. The limitation appears in loop level obligations such as global plan maintenance, residual continuity, and regression obligation retention rather than in per unit code generation alone.

3.2 RQ2: How Do Coding Agent Loops Maintain Plan, Code, and Test State?

Table 3 uses recorded loop traces to compare evaluated loops with the source recovered prerequisite DAG along planning, implementation, and testing axes. The runtime permits any topological execution order consistent with the DAG, while the planning diagnostics score source evidenced relations and layers rather than reproduction of a historical total order. **(1) Planning: an internal dependency gap.** Recorded plans recover partial dependency structure for long horizon routing, omitting many source evidenced pull request level prerequisites in the reference DAG. Edge F1 and Layer ρ remain far from perfect agreement with the source recovered prerequisite DAG for every evaluated loop, with lower values for open loop implementations than for closed ones. Critical Path

Table 3: Planning, implementation, and testing loop trace metrics.

Loop	Planning fidelity				Implementation		Testing		
	Edge F1	Layer ρ	CPR	WR	PatchLen	Jacc	#T	F2P	Reg↓
<i>Closed source loop implementations</i>									
Claude Code	0.71	0.65	0.31	1.08	1.58	0.62	28	0.47	7.11%
Codex	0.67	0.61	0.33	1.14	1.71	0.64	24	0.44	4.83%
GitHub Copilot	0.58	0.52	0.41	0.92	1.83	0.66	22	0.41	6.91%
<i>Open source loop implementations</i>									
OpenHands	0.39	0.33	0.85	0.39	2.31	0.73	16	0.34	2.46%
SWE-agent	0.37	0.31	0.88	0.36	2.36	0.74	15	0.33	2.18%
mini-swe-agent	0.27	0.22	0.97	0.24	2.54	0.76	11	0.28	0.24%

Ratio (CPR) and Width Ratio (WR) reveal how missing prerequisites reshape the planned concurrency budget. Closed source loop implementations organize execution through a tree shaped concurrent structure and stay close to the concurrency budget implied by the reference DAG, whereas open source loop implementations driven by linear control flow form a near chain. Width Ratio above unity for closed loop implementations is better interpreted as overparallelization than as planning fidelity beyond the source recovered reference DAG. A limited share of those branches reflects valid alternative decomposition, while most parallelize serial requirements absent from the recorded plan.

(2) Implementation: patch surplus over a comparable token base. On units that the evaluated configuration eventually resolves, PatchLen exceeds the gold reference by a consistent margin while token overlap with gold remains moderate. Candidate patches are therefore often longer than the reference while preserving enough token overlap to remain comparable to it. Appendix L provides a diff level account of where the extra lines come from. **(3) Testing: a maintenance gap over the long horizon.** Recorded loop traces contain sparse test authoring as long horizon work unfolds, leaving earlier obligations with limited agent authored regression protection against later edits. Test count (#T) is markedly lower than that of the native suite for every evaluated loop. Fail to pass yield (F2P) on authored tests remains acceptable after internal validation before commit. Regression (Reg) captures the downstream cost of this sparse maintenance. Deployed loop implementations exhibit several percentage points of regression over previously satisfied obligations. The lower apparent Regression Rate of lighter implementations reflects the limited number of passing obligations eligible for regression measurement. Long horizon loop engineering evaluation derives loop trace metrics from the running state of the evaluated loop, recording where plan, code, and test state are maintained, rerouted, or degraded.

RQ2: The long horizon gap centers on loop state discipline. Recorded loop traces capture missing dependencies, inflated patches, and sparse tests, linking performance to how plan, code, and test state are maintained across the run.

3.3 RQ3: Which Loop Engineering Factors Shape Long Horizon Execution?

RQ3 studies objective persistence, residual routing, context renewal, and regression pressure. We compare four representative loop runs. *Codex goal mode* keeps a persistent objective inside the Codex loop (OpenAI, 2026a). *Claude Code goal mode* maintains a session condition checked after each turn (Anthropic, 2026a). *Claude dynamic workflows* launch task specific workers with narrower local contexts (Anthropic, 2026c,b). The Ralph loop starts a fresh invocation for residual work (Huntley, 2025, 2026). Table 4 reports context-budget rounds, regression events per run, and Resolve Rate. A context-budget round is the initial context or a segment opened by the implementation compaction policy or by an external restart. Table 5 shows the compaction-budget sensitivity. Appendix N.1 gives the definitions and run coverage.

(1) Context renewal differs across loop implementations. After counting each compaction interval as a round, dynamic workflows record 97.96 context-budget rounds per run, compared with 34.69 for Claude Code goal mode, 32.76 for Codex goal mode, and 13.24 for Ralph. The dynamic workflow

Table 4: Loop trace metrics.

Loop Implementation	Rounds	Reg/run	RR (%)
Codex goal mode	32.76	0.34	20.59
Claude Code goal mode	34.69	0.13	17.65
Claude dynamic workflows	97.96	0.36	24.11
Ralph loop	13.24	0.17	7.84

Figure 5: Compaction sensitivity.

Codex	43.6	32.8	26.3
Claude	46.2	34.7	27.8
Dynamic	130.6	98.0	78.4
Ralph	17.2	13.2	10.8
	0.75x	default	1.25x
	Trigger budget		

run has the highest Resolve Rate at 24.11%, while Ralph reaches 7.84%. These values describe the evaluated configurations.

(2) Context-budget renewal does not remove regression pressure. Dynamic workflows record 0.36 regression events per run despite distributing work across narrower worker contexts. Codex goal mode records 0.34, Claude Code goal mode 0.13, and Ralph 0.17. Dynamic routing therefore reduces dependence on a single growing transcript, but completed obligations still require explicit state tracking when workers return. Appendix N.3 states the trace scope for worker local contexts.

RQ3: Loop engineering changes how context is renewed and how residual work is routed. Dynamic workflows achieve the strongest observed outcome, while regression events remain visible across all four loop profiles.

4 Related Work

Prior coding benchmarks establish reliable end state scoring for code changes, but they offer limited evidence about the loops that sustain long horizon execution. Function level code generation benchmarks (Chen et al., 2021) pioneered programmatic evaluation on localized snippets, while repository level benchmarks centered on SWE-bench (Jimenez et al., 2023a) package work as issue resolution and emphasize end state pass rate. Feature and evolution benchmarks (Zhou et al., 2026) enlarge task scope, and broader long horizon benchmarks (Kwa et al., 2025) extend duration, repository scale, or interaction depth. Across these families, prerequisite state, residual work routing, and regression obligations generally remain outside the executable scoring contract. Success is therefore assigned to submitted artifacts rather than to the loop state that produced them. Modern coding agent infrastructure (Anthropic, 2025d; OpenAI, 2026a; Anthropic, 2026c; Huntley, 2026) increasingly relies on persistent objectives, worker orchestration, restarts, and context management. Loop implementation therefore becomes part of the evaluated behavior, while current benchmark designs provide limited evidence about how evaluated loops preserve state and route residual work. Software engineering work on change impact and release planning (Lehnert, 2011) treats dependency structure as an explicit artifact. LOOPS BENCH draws on this view by converting dependency structure into executable obligations for coding agent loop evaluation. Appendix A provides the full grouping.

5 Conclusion

LOOPS BENCH makes loop engineering a measurable dimension of coding agent evaluation alongside end state scoring. It formalizes long horizon tasks as dependency DAGs, releases tests along graph edges, and retains completed unit tests as persistent regression obligations while leaving execution order to the evaluated loop. This design records loop traces that capture residual work routing, state continuity, and obligation retention across iterations. Across frontier LLMs and loop implementations, the strongest configuration resolves 25.00% of tasks, regression events remain visible across all four RQ3 loop profiles, and recorded plans recover only part of the source recovered prerequisite DAG; the diagnostic does not require reproduction of a particular historical total order. These results support evaluating long horizon coding agents with joint attribution to model choice and loop configuration, especially residual work handling, structured state, and regression obligation retention. Scope conditions, source assumptions, and version pinning caveats are stated in the Limitations section.

Limitations

The recovered DAG is a lower bound on prerequisite structure rather than a complete causal graph of development. Edge admission uses four source evidenced patterns and a hot file denylist. The resulting depth and width statistics are therefore conservative. It may miss implicit dependencies in configuration, data formats, build systems, or cross service behavior. The fixed DAG is an evaluation contract: unit boundaries and reference patches are nonunique, and exploratory redesigns that replace earlier requirements fall outside its scope. The three pipelines admit tasks when commit history, curriculum decomposition, or citation closure yields verifiable evidence, narrowing the current pool to sources with auditable prerequisite evidence and leaving mobile, frontend heavy, and hardware adjacent projects outside scope. Reported correctness metrics are tied to the released test suite, including TPR, Resolve Rate, and Regression Rate, and therefore measure executable obligations rather than full semantic equivalence. Hidden checkpoint state can make regressions reflect limited feedback as well as regression discipline. Claude Code is used only offline; evaluated loops share one contract and receive no construction transcript, gold solution, or active frontier. Model assisted materialization may still bias wording, boundaries, and tests, while public sources leave contamination risk. LOOPS BENCH records coding agent loop execution through traces made accessible by current loop implementations and evaluation adapters. Internal events outside these interfaces cannot be measured directly. As goal modes, dynamic workflows, compaction policies, and external loop implementations evolve, LOOPS BENCH provides a loop engineering evaluation interface with replaceable adapters and version pinned run metadata. Evaluation is costly, and proprietary outputs may drift. We will release tasks, tests, pinned environments, construction and evaluation code, prompts, configurations, traces, checksums, and a smoke test subset to reproduce the contract, though not necessarily exact outputs. LOOPS BENCH measures one executable contract, not general software engineering ability or deployment readiness; generated code still requires human review, security testing, and licensing checks.

Compliance, Safety, and Data Statement

LOOPS BENCH is constructed from public open source repositories, public course artifacts, and publicly released research code. It does not use personal data, customer data, or private production data. Generated benchmark instances are derived from source artifacts and should preserve the applicable source licenses, attribution requirements, and redistribution constraints. The construction pipeline may reintroduce historical defects, including security relevant defects, as part of benchmark generation. These instances are intended for research evaluation rather than deployment. We assume source histories and release artifacts are benign and trusted. Malicious, poisoned, or intentionally deceptive repositories are outside the scope of this benchmark.

References

- Alibaba Cloud. Alibaba cloud unveils strategic roadmaps for the next generation of AI innovations. <https://alizila.oss-us-west-1.aliyuncs.com/uploads/2025/09/Media-Release-Alibaba-Cloud-Unveils-Strategic-Roadmaps-for-the-Next-Generation-AI-Innovations-0924.pdf>, 2025. Includes Qwen3-Max. Accessed 2026-05-16.
- Alibaba Group. Alibaba open-sources Qwen3.5, a natively multimodal model built for high-efficiency inference. <https://www.alibabagroup.com/document-1960233590314762240>, 2026. Identifies Qwen3.5-397B-A17B as Qwen3.5-Plus. Accessed 2026-05-16.
- Anthropic. Claude 4 system card. <https://assets.anthropic.com/m/6c940a1b69ed6a1c/original/Claude-4-System-Card.pdf>, 2025a. Covers Claude Opus 4 and Claude Sonnet 4. Accessed 2026-05-16.
- Anthropic. Introducing Claude Opus 4.5. <https://www.anthropic.com/news/claude-opus-4-5>, 2025b. Accessed 2026-05-16.
- Anthropic. Introducing Claude Sonnet 4.5. <https://www.anthropic.com/news/claude-sonnet-4-5>, 2025c. Accessed 2026-05-16.

- Anthropic. Claude code, 2025d. URL <https://www.anthropic.com/claude-code>. Anthropic product page.
- Anthropic. Keep claude working toward a goal – claude code docs, 2026a. URL <https://code.claude.com/docs/en/goal>. Accessed: 2026-06-14.
- Anthropic. Create custom subagents – claude code docs, 2026b. URL <https://code.claude.com/docs/en/sub-agents>. Accessed: 2026-06-14.
- Anthropic. Orchestrate subagents at scale with dynamic workflows – claude code docs, 2026c. URL <https://code.claude.com/docs/en/workflows>. Accessed: 2026-06-14.
- Anthropic. Introducing Claude Opus 4.7. <https://www.anthropic.com/news/claude-opus-4-7>, 2026d. Accessed 2026-05-16.
- Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, et al. Graph of thoughts: Solving elaborate problems with large language models. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, pages 17682–17690, 2024.
- Nicholas Carlini. Building a C compiler with a team of parallel claudes, 2026. URL <https://www.anthropic.com/engineering/building-c-compiler>. Anthropic Engineering blog.
- Guoxin Chen, Fanzhe Meng, Jiale Zhao, Minghao Li, Daixuan Cheng, Huatong Song, Jie Chen, Yuzhi Lin, Hui Chen, Xin Zhao, et al. Beyondswe: Can current code agent survive beyond single-repo bug fixing? *arXiv preprint arXiv:2603.03194*, 2026.
- Haorui Chen, Chengze Li, and Jia Li. Featbench: Evaluating coding agents on feature implementation for vibe coding. *arXiv preprint arXiv:2509.22237*, 2025.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Cognition. Introducing devin, 2026. URL <https://docs.devin.ai/get-started/devin-intro>. Devin Docs.
- Daniel Alencar da Costa, Shane McIntosh, Weiyi Shang, Uirá Kuber, Yasutaka Kamei, and Ahmed E. Hassan. A framework for evaluating the results of the SZZ approach for identifying bug-introducing changes. In *IEEE Transactions on Software Engineering*, volume 43, pages 641–657. IEEE, 2016.
- DeepSeek-AI. DeepSeek-V4-Pro model card. <https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro>, 2026. Accessed 2026-05-16.
- Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, et al. Swe-bench pro: Can ai agents solve long-horizon software engineering tasks? *arXiv preprint arXiv:2509.16941*, 2025.
- Jingzhe Ding, Shengda Long, Changxin Pu, Huan Zhou, Hongwan Gao, et al. Nl2repo-bench: Towards long-horizon repository generation evaluation of coding agents. *arXiv preprint arXiv:2512.12730*, 2025. URL <https://arxiv.org/abs/2512.12730>.
- Nicholas Edwards, Yukyung Lee, Yujun Audrey Mao, Yulu Qin, Sebastian Schuster, and Najoung Kim. REXBench: Can coding agents autonomously implement AI research extensions? *arXiv preprint arXiv:2506.22598*, 2025.
- Yukang Feng, Jianwen Sun, Zelai Yang, Jiaxin Ai, Chuanhao Li, et al. Longcli-bench: A preliminary benchmark and study for long-horizon agentic programming in command-line interfaces. *arXiv preprint arXiv:2602.14337*, 2026. URL <https://arxiv.org/abs/2602.14337>.
- Dhruv Gautam, Spandan Garg, Jinu Jang, Neel Sundaresan, and Roshanak Zilouchian Moghaddam. Refactorbench: Evaluating stateful reasoning in language agents through code. *arXiv preprint arXiv:2503.07832*, 2025.

- GitHub. Github copilot cloud agent, 2026. URL <https://docs.github.com/en/copilot/how-to/use-copilot-agents/cloud-agent>. GitHub Docs.
- Google DeepMind. Gemini 3.1 Pro model card. <https://deepmind.google/models/model-cards/gemini-3-1-pro>, 2026. Accessed 2026-05-16.
- Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, et al. Measuring coding challenge competence with apps. *arXiv preprint arXiv:2105.09938*, 2021.
- Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. Metagpt: Meta programming for a multi-agent collaborative framework. In *The twelfth international conference on learning representations*, 2023.
- Geoffrey Huntley. Ralph wiggum as a “software engineer”, 2025. URL <https://ghuntley.com/ralph/>. Accessed: 2026-05-07.
- Geoffrey Huntley. Everything is a ralph loop, 2026. URL <https://ghuntley.com/loop/>. Accessed: 2026-06-14.
- Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770*, 2023a.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770*, 2023b. URL <https://arxiv.org/abs/2310.06770>.
- Thomas Kwa, Ben West, Joel Becker, Amy Deng, Katharyn Garcia, Max Hasin, Sami Jawhar, Megan Kinniment, Nate Rush, Sydney Von Arx, et al. Measuring ai ability to complete long tasks. *arXiv preprint arXiv:2503.14499*, 352, 2025.
- Steffen Lehnert. A taxonomy for software change impact analysis. *Proceedings of the 12th International Workshop on Principles of Software Evolution and the 7th Annual ERCIM Workshop on Software Evolution*, pages 41–50, 2011.
- Bowen Li, Wenhan Wu, Ziwei Tang, Lin Shi, John Yang, Jinyang Li, Shunyu Yao, Chen Yin, Yixin Liu, Hao Peng, et al. DevBench: A comprehensive benchmark for software development. *arXiv preprint arXiv:2403.08604*, 2024.
- Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *Advances in neural information processing systems*, 36:21558–21572, 2023.
- Yuxiang Liu et al. SWE-bench-M: A benchmark for mobile software engineering agents. *arXiv preprint*, 2025.
- Mike A Merrill, Alexander G Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E Kelly Buchanan, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *arXiv preprint arXiv:2601.11868*, 2026.
- Samuel Miserendino, Michele Wang, Tejal Patwardhan, and Johannes Heidecke. Swe-lancer: Can frontier llms earn \$1 million from real-world freelance software engineering? In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 44412–44450, 2025. URL <https://proceedings.mlr.press/v267/miserendino25a.html>.
- Moonshot AI. Kimi K2.6: From code to creation, from one to many. <https://www.kimi.com/ai-models/kimi-k2-6>, 2026. Use for the RQ1 table entry named Kimi-2.6 unless the intended model is changed. Accessed 2026-05-16.

- OpenAI. GPT-4o system card. <https://cdn.openai.com/gpt-4o-system-card.pdf>, 2024a. Accessed 2026-05-16.
- OpenAI. Introducing swe-bench verified, 2024b. URL <https://openai.com/index/introducing-swe-bench-verified/>. OpenAI blog.
- OpenAI. Introducing codex, 2025a. URL <https://openai.com/index/introducing-codex/>. OpenAI product announcement.
- OpenAI. Introducing GPT-5. <https://openai.com/index/introducing-gpt-5/>, 2025b. Accessed 2026-05-16.
- OpenAI. Update to GPT-5 system card: GPT-5.2. https://cdn.openai.com/pdf/3a4153c8-c748-4b71-8e31-aecbde944f8d/oai_5_2_system-card.pdf, 2025c. Accessed 2026-05-16.
- OpenAI. Prompting – codex, 2026a. URL <https://developers.openai.com/codex/prompting>. Accessed: 2026-06-14.
- OpenAI. Introducing GPT-5.3 Instant, GPT-5.4 Thinking, and GPT-5.4 Pro. <https://academy.openai.com/en/home/resources/latest-model>, 2026b. Accessed 2026-05-16.
- OpenAI. Introducing GPT-5.5. <https://openai.com/index/introducing-gpt-5-5/>, 2026c. Accessed 2026-05-16.
- OpenAI. Why swe-bench verified no longer measures frontier coding capabilities, 2026d. URL <https://openai.com/index/why-we-no-longer-evaluate-swe-bench-verified/>. OpenAI research note.
- Gabriel Orlanski, Devjeet Roy, Alexander Yun, Changho Shin, Alex Gu, Albert Ge, Dyah Adila, Frederic Sala, and Aws Albarghouthi. Slopcodebench: Benchmarking how coding agents degrade over long-horizon iterative tasks. *arXiv preprint arXiv:2603.24755*, 2026.
- Charles Packer, Vivian Fang, Shishir_G Patil, Kevin Lin, Sarah Wooders, and Joseph_E Gonzalez. Memgpt: towards llms as operating systems. 2023.
- Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with swe-gym. *arXiv preprint arXiv:2412.21139*, 2024. URL <https://arxiv.org/abs/2412.21139>.
- Qwen Team. Qwen2.5 technical report. <https://arxiv.org/abs/2412.15115>, 2024. Accessed 2026-05-16.
- Qwen Team. Qwen3.6-Plus: Towards real world agents. <https://qwen.ai/blog?id=qwen3.6>, 2026. Accessed 2026-05-16.
- Günther Ruhe. *The Art and Science of Software Release Planning*. Springer, 2005.
- Atharva Sehgal, James Hou, Swarat Chaudhuri, Jennifer J Sun, and Yisong Yue. Formulacode: Evaluating agentic superoptimization on large codebases. In *ICML 2025 Workshop on Programmatic Representations for Agent Learning*.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023.
- Zachary S Siegel, Sayash Kapoor, Nitya Nagdir, Benedikt Stroebel, and Arvind Narayanan. Core-bench: Fostering the credibility of published research through a computational reproducibility agent benchmark. *arXiv preprint arXiv:2409.11363*, 2024.
- Atharv Sonwane, Eng-Shen Tu, Wei-Chung Lu, Claas Beger, Carter Larsen, Debjit Dhar, Simon Alford, Rachel Chen, Ronit Pattanayak, Tuan Anh Dang, et al. Omnicode: A benchmark for evaluating software engineering agents. *arXiv preprint arXiv:2602.02262*, 2026.

- Alvin W Tan, Sunny Yu, Bria Long, Wanjing Anya, Tonya Murray, Rebecca D Silverman, Jason D Yeatman, and Michael C Frank. Devbench: A multimodal developmental benchmark for language learning. *Advances in Neural Information Processing Systems*, 37:77445–77467, 2024.
- Minh VT Thai, Tue Le, Dung Nguyen Manh, Huy Phan Nhat, and Nghi DQ Bui. Swe-evo: Benchmarking coding agents in long-horizon software evolution scenarios. *arXiv preprint arXiv:2512.18470*, 2025.
- Qiang Tu and Michael W. Godfrey. The evolution of large C programs. In *Science of Computer Programming*, volume 45, pages 203–225. Elsevier, 2002.
- Harold Valdivia Garcia and Emad Shihab. Characterizing and predicting blocking bugs in open source projects. *Journal of Systems and Software*, 143:44–58, 2018.
- Sanidhya Vijayvargiya, Xuhui Zhou, Akhila Yerukola, Maarten Sap, and Graham Neubig. Ambig-swe: Interactive agents to overcome underspecificity in software engineering. *arXiv preprint arXiv:2502.13069*, 2025.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. Openhands: An open platform for ai software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.
- Hjalmar Wijk, Tao Lin, Joel Becker, Sami Jawhar, et al. Re-bench: Evaluating frontier ai r&d capabilities of language model agents against human experts. In *ICML 2025 Workshop / OpenReview version*, 2025. URL <https://openreview.net/forum?id=3rB0bVU6z6>.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversations. In *First conference on language modeling*, 2024.
- xAI. Grok 4.1 Fast Reasoning. <https://docs.x.ai/developers/models/grok-4-1-fast-reasoning>, 2026. Accessed 2026-05-16.
- Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying llm-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024. URL <https://arxiv.org/abs/2407.01489>.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024a.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R. Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024b. URL <https://arxiv.org/abs/2405.15793>.
- John Yang, Carlos E Jimenez, Alex L Zhang, Kilian Lieret, Joyce Yang, Xindi Wu, Ori Press, Niklas Muennighoff, Gabriel Synnaeve, Karthik R Narasimhan, et al. Swe-bench multimodal: Do ai systems generalize to visual software domains? *arXiv preprint arXiv:2410.03859*, 2024c.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36:11809–11822, 2023.
- Z.AI. GLM-5.1 overview. <https://docs.z.ai/guides/llm/glm-5.1>, 2026. Accessed 2026-05-16.
- Daoguang Zan, Zhirong Huang, Wei Liu, Hanwu Chen, Linhao Zhang, Shulin Xin, Lu Chen, Qi Liu, Xiaojian Zhong, Aoyan Li, et al. Multi-swe-bench: A multilingual benchmark for issue resolving. *arXiv preprint arXiv:2504.02605*, 2025.

Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. Autocoderover: Autonomous program improvement. *arXiv preprint arXiv:2404.05427*, 2024. URL <https://arxiv.org/abs/2404.05427>.

Qixing Zhou, Jiacheng Zhang, Haiyang Wang, Rui Hao, Jiahe Wang, Minghao Han, Yuxue Yang, Shuzhe Wu, Feiyang Pan, Lue Fan, et al. Featurebench: Benchmarking agentic coding for complex feature development. *arXiv preprint arXiv:2602.10975*, 2026.

Yuxuan Zhu, Tengjun Jin, Yada Pruksachatkun, Andy Zhang, Shu Liu, Sasha Cui, Sayash Kapoor, Shayne Longpre, Kevin Meng, Rebecca Weiss, et al. Establishing best practices for building rigorous agentic benchmarks. *arXiv preprint arXiv:2507.02825*, 2025.

A Related Work for Loop Engineering Evaluation

Table 5 groups the full set of prior work surveyed in Section 4 by the evaluation evidence each cluster supports and by its relation to loop engineering evaluation. Section 4 cites the lead exemplar of each cluster, while this appendix records additional references without expanding main text citation density.

Table 5: Prior work related to loop engineering evaluation.

Cluster	References (representative + extensions)
Function level code generation. Report localized unit test success, without state retention or residual routing over long horizons.	Chen et al. (2021); Hendrycks et al. (2021); Jain et al. (2024); Liu et al. (2023)
SWE-bench family. Repository level issue tasks that primarily report end state success for a submitted patch.	Jimenez et al. (2023a,b); Zan et al. (2025); Yang et al. (2024c); Liu et al. (2025); Deng et al. (2025); OpenAI (2024b); Miserendino et al. (2025); Pan et al. (2024); Wijk et al. (2025); Sonwane et al. (2026); Sehgal et al.; Tan et al. (2024)
SWE-bench audits. Document contamination and overfitting risks in verified issue resolution variants.	OpenAI (2026d); Zhu et al. (2025)
Feature and evolution. Broaden task scope while generally leaving prerequisite structure outside executable obligations that measure evaluated loops.	Zhou et al. (2026); Chen et al. (2025); Thai et al. (2025); Gautam et al. (2025); Orlanski et al. (2026); Feng et al. (2026); Ding et al. (2025)
Interaction and trace benchmarks. Extend beyond one submitted change while generally leaving completed units outside active regression obligations during the recorded loop trace.	Kwa et al. (2025); Siegel et al. (2024); Li et al. (2024); Merrill et al. (2026); Edwards et al. (2025); Chen et al. (2026); Vijayvargiya et al. (2025)
Loop implementations and infrastructure. Systems and frameworks that instantiate coding agent loops through prompts, tools, memory, execution policies, and adapters, forming the system boundary studied by loop engineering evaluation.	Anthropic (2025d); OpenAI (2025a); Cognition (2026); Wang et al. (2024); Yang et al. (2024a,b); Zhang et al. (2024); Xia et al. (2024); GitHub (2026); Huntley (2025)
Loop engineering infrastructure. Describe infrastructure patterns for persistent goals, external loops, dynamic worker orchestration, and subagent routing, but do not define a benchmark over dependency structured obligations.	OpenAI (2026a); Anthropic (2026a,c,b); Huntley (2026)
Reasoning frameworks. Provide planning, memory, reflection, and context management primitives that loop implementations build on, rather than executable loop trace evidence.	Yao et al. (2023); Besta et al. (2024); Shinn et al. (2023); Hong et al. (2023); Wu et al. (2024); Wang et al. (2023); Packer et al. (2023)
Software engineering prerequisites. Study dependency structure, PR ordering, and release planning, providing source concepts that LOOPS-BENCH converts into executable obligations and loop trace metrics.	Lehnert (2011); Tu and Godfrey (2002); Costa et al. (2016); Valdivia Garcia and Shihab (2018); Ruhe (2005)

B Task Pool Statistics for Loop Engineering Evaluation

The 112 LOOPS-BENCH tasks decompose into 29 PR Sequences (Infra and Data, Backend, Frontend), 57 Course Labs (Game Engines, Systems, Apps and ML), and 26 Research Evolutions (Analysis and Verification, Systems and Networking, Compiler with Database and ML). This source mix spans 26 leaf subcategories and covers software work with different dependency structures, routing burdens, and verification obligations.

Tables 6 and 7 report per task structural statistics for all 112 LOOPS-BENCH tasks, including repository scale (#Files, #LoC), unit count (#Num development units), task structure summaries (average

#Modules, average DAG edges, average dependency depth), and gold implementation targets (average #Lines, #Files, #LoC). These statistics describe the work graph and implementation scope used to quantify routing burden and obligation retention pressure in loop engineering evaluation. The split into two tables is for typesetting, and the row sets are disjoint and cover the full task pool.

Table 6: Structural statistics for LOOPSBENCH tasks, Part 1.

Repository			Units	Task structure			Implementation target		
Task	#Files	#LoC	#Num	#Mod	#Edges	Depth	Avg. LoC	Max LoC	Width
task_ClickHouse_seg07	3.3k	511k	817	30	2121	34	625	71.4k	404
task_NodeBB_seg01	1.7k	396k	138	17	640	27	2.9k	64.2k	28
task_NodeBB_seg03	4.1k	294k	382	22	130	10	770	35.5k	166
task_NodeBB_seg05	4.6k	130k	355	20	127	10	365	11.3k	310
task_TypeScript_seg01	14.4k	379k	406	18	1119	47	934	95.6k	304
task_afl_fuzzing	11	5142	5	5	4	2	1.0k	1400	4
task_alloy_formal_modeling	18	6457	4	4	4	4	1.6k	1845	1
task_angr_concolic_execution	14	5963	5	5	5	3	1.2k	1295	3
task_ansible_seg11	5.1k	45.8k	319	20	704	17	144	11.8k	134
task_ansible_seg13	5.8k	25.6k	111	16	59	5	231	6010	69
task_architecture_riscv_labs	420	1130	15	15	7	3	75.3	273	9
task_boogie_dafny	21	7025	3	3	2	3	2.3k	2811	1
task_calcite_query_optimization	9	6411	3	3	2	2	2.1k	2741	2
task_cocos2dx_navmesh_medium	4.5k	5207	14	14	12	4	372	1143	9
task_cocos2dx_physics2d_medium	4.2k	4700	18	18	21	7	261	503	9
task_compiler	11	1057	27	27	31	8	39.1	107	6
task_compiler_cmm_mips	29	2688	9	9	7	4	299	1839	6
task_compiler_fdmj_llvm	89	1304	7	7	5	4	186	472	2
task_compiler_rustlike_rv32	97	2079	6	6	6	4	346	606	2
task_compiler_sysy_rust	21	1906	6	6	5	4	318	591	2
task_cpachecker	26	80	3	3	2	3	26.7	34	1
task_cs61b_extra_java_bundle	218	2837	22	22	13	3	129	308	10
task_cvc_smt_solver	31	8013	3	3	2	3	2.7k	3065	1
task_datastructs_java	219	1977	15	15	4	2	132	330	11
task_db_nanodb	417	1522	9	9	8	3	169	774	4
task_db_query_optimizer_labs	70	1005	26	26	41	26	38.7	111	1
task_db_storage_index_labs	240	2398	10	10	11	7	240	1374	3
task_dbcompiler	21	1217	24	24	21	10	50.7	161	5
task_dcqcn_rdma_evolution	2.3k	31.2k	4	11	3	4	7.8k	15.7k	1
task_deep_learning_three_assignments	91	1365	16	16	10	4	85.3	202	7
task_deep_rl_coursework	203	3285	19	19	23	6	173	811	6
task_dist_sysRAFT-go	89	1173	4	4	5	3	293	581	2
task_distributed_fs_chfs	2.1k	2875	12	12	15	7	240	1195	3
task_django_seg01	6.2k	16.5k	209	16	476	21	78.8	1687	45
task_django_seg05	6.6k	15.9k	221	16	357	15	71.8	1690	108
task_django_seg11	6.8k	10.6k	165	17	213	12	64.0	718	80
task_doop_souffle_static_analysis	17	6342	5	5	4	3	1.3k	1458	2
task_ds_five_assignments	27	4914	5	5	0	1	983	1231	5
task_echarts_seg01	1.3k	19.9k	52	18	140	14	382	3903	15
task_echarts_seg04	1.4k	26.3k	53	20	153	13	496	12.6k	22
task_echarts_seg05	1.5k	14.2k	43	20	140	12	330	3271	20
task_echarts_seg07	1.5k	9980	65	10	223	13	154	2093	29
task_echarts_seg08	1.6k	28.3k	77	11	340	20	367	4742	32
task_echarts_seg09	1.6k	6779	46	11	128	12	147	1037	26
task_echarts_seg10	1.6k	17.3k	47	24	72	9	367	2229	30
task_echarts_seg11	1.6k	12.4k	62	38	215	17	200	1679	31
task_echarts_seg13	1.9k	6123	46	34	60	7	133	1509	25
task_evosuite_test_generation	17	8216	6	6	6	3	1.4k	1434	3
task_frama_c	21	4659	3	3	2	3	1.6k	2095	1
task_framework_seg04	1.4k	130k	383	18	766	22	339	15.6k	147
task_gem5_architecture_simulation	4	6004	4	4	3	2	1.5k	1661	3
task_godot_bootstrap_medium	13.8k	5020	14	14	15	8	359	1394	3
task_godot_navigation2d_medium	13.8k	5239	11	11	12	5	476	1261	7
task_graph_processing	6	5399	3	3	2	3	1.8k	1819	1
task_graphics_raster_2d	17	1305	12	12	17	4	108.8	529	7
task_graphics_scotty3d	390	1199	8	8	2	3	150	276	6

Table 7: Structural statistics for LOOPSBENCH tasks, Part 2.

Repository			Units	Task structure			Implementation target		
Task	#Files	#LoC	#Num	#Mod	#Edges	Depth	Avg. LoC	Max LoC	Width
task_hadoop_seg05	36.3k	533k	318	16	597	16	1.7k	103k	142
task_java_ds_coursework	96	3997	17	17	10	4	235	522	6
task_jenkins_seg02	23.6k	494k	84	11	167	16	5.9k	43.5k	40
task_jenkins_seg05	12.3k	63.4k	44	7	40	7	1.4k	13.1k	21
task_jpf_symbolic_execution	1.2k	229k	3	3	2	3	76.4k	126k	1
task_ligra_graph_evolution	57	14.8k	2	8	1	2	7.4k	7990	1
task_linux011_kernel_hacks	137	2690	13	13	15	6	207	724	5
task_llvm_alive_verification	556	32.8k	3	3	2	2	10.9k	27.0k	2
task_lsm_vector_store_four_stage	1.2k	1841	6	6	6	4	307	1832	2
task_mininet_network_simulation	10	6240	5	5	5	3	1.2k	1339	3
task_ml_four_assignments	42	2002	10	10	7	3	200	651	5
task_mlir_circuit_ir	10	5037	3	3	2	3	1.7k	2237	1
task_mlp_numpy_coursework	64	1024	7	7	6	3	146	329	4
task_mlsys_autodiff_pa1	18	1263	20	20	34	10	63.1	360	3
task_monogame_math2d_medium	2.1k	5002	12	12	17	4	417	1393	4
task_monogame_transform3d_medium	2.1k	4481	8	8	12	4	560	2318	2
task_mysql_sql_labs	48	1093	9	9	7	3	121	413	5
task_nandteris	256	1020	6	6	8	5	170	278	2
task_navidrome_seg02	364	52.1k	64	11	9	3	814	11.0k	49
task_navidrome_seg04	570	29.1k	56	12	16	5	520	13.1k	44
task_navidrome_seg05	627	50.1k	80	12	14	4	626	17.4k	59
task_navidrome_seg07	752	7595	44	12	4	3	173	1448	37
task_net_os_labs	54	2019	18	18	10	6	112	146	9
task_networks_proxy_rdt	48	1007	22	22	15	3	45.8	216	14
task_node_seg03	38.8k	126k	321	24	358	15	393	49.6k	210
task_ns3_network_simulation	3.2k	337k	2	2	1	2	168.3k	190k	1
task_numpy_dl_aics_style	176	1001	20	20	23	12	50.0	113	3
task_opengl_glfw_labs	1.4k	1259	8	8	7	8	157	274	1
task_os_c_fs_labs	28	1230	7	7	1	2	176	639	6
task_os_distributed_practicals	19	1049	5	5	0	1	210	595	5
task_os_labs_asm_kernel	780	14.3k	11	11	9	5	1.3k	3874	4
task_panda3d_collision_hard	6.7k	4813	14	14	14	5	344	903	6
task_panda3d_pgraph_medium	6.6k	3767	12	12	11	4	314	895	5
task_perses_program_reduction	21	5853	5	5	5	3	1.2k	1283	3
task_rails_seg09	4.8k	11.4k	122	16	90	7	93.7	1146	78
task_raylib_shapes2d_medium	1.4k	5083	10	10	18	5	508	1491	5
task_raylib_textures_hard	1.4k	6359	11	11	26	5	578	1548	5
task_rdma_ns3_evolution	2.3k	590	3	12	2	3	197	527	1
task_riscv_cpu_six_labs	305	3429	6	6	6	3	572	720	4
task_riscv_cpu_sv	133	1875	12	12	17	6	156	449	5
task_riscv_cpu_verilog_three	56	1157	6	6	6	5	193	275	2
task_riscv_os_multiproject	619	4254	7	7	5	6	608	2006	2
task_sat_minisat_glucose	29	17.3k	2	9	1	2	8.7k	16.9k	1
task_security_labs	84	1056	25	25	23	6	42.2	103	7
task_sicp_py_scheme_sql	696	3135	22	22	25	13	142	698	5
task_sklearn_ml_toolchain	4	5252	3	3	3	3	1.8k	2445	1
task_soot_taint_analysis	10	5212	3	3	2	3	1.7k	1928	1
task_spring_course_mgmt_fullstack	19.8k	1116	10	10	15	4	112	263	3
task_sql_engine_myjql	44	1263	5	5	5	4	253	524	2
task_stream_mining	5	5245	5	5	5	3	1.0k	1241	2
task_systems_csapp_labs	101	1289	6	6	1	2	215	323	5
task_systems_four_labs	342	1913	10	10	5	3	191	623	6
task_tcp_course_stack	48	1405	2	2	1	2	702	1192	1
task_x86_os_four_labs	128	2229	9	9	11	6	248	600	2
task_xjqkl	26	1307	6	6	9	4	218	632	3
task_z3_fixedpoint_verification	11	6074	5	5	5	4	1.2k	1345	2

C Task Schema for Loop Engineering Evaluation



Each task is materialized as a self contained directory for loop engineering evaluation, separating the work graph, presented requirements, executable obligations, and reference solution. This separation maintains a distinction between the evaluated loop’s input contract and the hidden artifacts used to validate obligations and compute loop trace metrics.

The Dockerfile and optional docker-compose.yaml pin a reproducible execution environment.

run-tests.sh is the checkpoint test entry point used by the evaluation runtime to adjudicate the current workspace state.

solution.sh and gold-patch.diff store the reference human solution used to validate executable obligations and anchor implementation comparisons.

task.yaml declares task metadata and per unit fields, including requirement text, file and symbol scope, dependency parents, and attached tests.

The tests/ subtree holds source derived and supplemented acceptance tests indexed to the unit they adjudicate.

The evaluation contract is uniform across the three task sources and separates the full requirement set presented to the evaluated loop from the active obligations counted at each checkpoint. The evaluation

On-disk layout of a single LOOPS BENCH task	
<code><repo>-<task-id>/</code>	<i>self-contained task bundle</i>
<code>Dockerfile</code>	<i>pinned base image, system deps, lockfile</i>
<code>docker-compose.yaml</code>	<i>tasks with multiple services</i>
<code>run-tests.sh</code>	<i>checkpoint test entry point for the evaluation runtime</i>
<code>solution.sh</code>	<i>driver that applies the reference solution</i>
<code>gold-patch.diff</code>	<i>reference human solution as a unified diff</i>
<code>task.yaml</code>	<i>global task metadata and DAG topology</i>
<code>units/</code>	<i>requirements and test bindings presented during evaluation</i>
<code>unit_001.yaml</code>	<i>requirement, scope, parents, attached tests</i>
<code>unit_002.yaml</code>	
<code>...</code>	
<code>unit_NNN.yaml</code>	
<code>tests/</code>	<i>source-derived and supplemented test files</i>
<code>unit_001/</code>	<i>tests adjudicating unit 001</i>
<code>unit_002/</code>	
<code>...</code>	
<code>unit_NNN/</code>	
<code>base/</code>	<i>entry-commit codebase the patch is applied to</i>

Figure 6: Schema of a materialized LOOPS BENCH task.

Table 8: Visibility contract used by LOOPS BENCH. Unit manifests and attached tests are presented up front, while checkpoint progress and scoring state remain evaluator only.

Evaluated-loop visible	Evaluator only
Global task instruction and unit manifests, including requirements, scopes, declared parents, and attached test bindings	Active ready frontier and newly released obligation state
Base repository, attached and project native tests, and outputs of commands initiated by the evaluated loop	Reference patches, gold solution, reference implementation, and offline construction transcripts
Tests authored and run by the evaluated loop	Checkpoint outcomes, accumulated regression obligations, and scoring state

runtime mounts the task directory into the container, presents *all* per unit requirement manifests up front, invokes `run-tests.sh` at committed checkpoints, and records the resulting test state vector $s_i^{(\tau)}$ used to compute the loop trace metrics in Appendix G. The *ready frontier* controls newly released obligations, while previously passed released tests remain active as regression obligations during later work.

D Materialized Task Example for Loop Engineering Evaluation

This appendix presents one materialized task and the loop trace metrics that LOOPS BENCH records during evaluation, making the schema in Section 2 and Appendix C concrete. The example is a PR Sequences task drawn from an actively maintained open source repository. It connects materialization steps to loop trace metrics, including commit history snapshotting, per pull request diff extraction by role, dependency routing, ready frontier test release, and regression obligation retention. Under the contamination protocol of Appendix E, identifiable tokens are redacted and marked as `<repo>`, `<unit-id>`, and `<author>`.

Native artifact. The task originates from a sequence of **9** merged pull requests against <repo> over **4.2 months**, touching **37** source files and **14** test files. The base codebase at the entry commit comprises **18.4k** LoC across **142** files. Each pull request provides a description, a per file diff, and a CI run record from the upstream test suite, giving source evidence for implementation units, prerequisite relations, and executable obligations.

Preprocessing output. After preprocessing, the nine pull requests decompose into **12** atomic candidate units. Three original PRs are split when their diffs touch disjoint feature surfaces, and two adjacent PRs are merged when the second repairs a regression introduced by the first and is therefore dependent on that PR. The materialization extractor produces the aggregated segments under the prescribed extraction procedure, and the segments pass the fixed acceptance standard on the first attempt.

Materialized task bundle. The candidate is rewritten into the uniform schema of Appendix C. The resulting on disk layout is:

Task bundle layout

```
<repo>-<unit-id>/
  Dockerfile
  docker-compose.yaml
  run-tests.sh
  solution.sh
  gold-patch.diff
  task.yaml
  tests/
    unit_001/...
    unit_002/...
    ...
    unit_012/...
```

The Dockerfile pins Python 3.11.6, three system packages absent from the upstream README but recovered from CI configuration, and a lockfile derived from the entry commit. The materialization stage verifies gold-patch.diff successfully after a single build attempt.

Per unit requirement. A representative unit's task.yaml entry (paraphrased and redacted) reads:

Per unit requirement (excerpt)

```
unit_id: <unit-id>
requirement: |
  Extend the streaming response handler so that
  partial chunks emitted before a terminating
  sentinel are buffered into a single payload
  delivered to downstream consumers, preserving
  ordering across concurrent producers.
file_scope:
  - src/stream/handler.py
  - src/stream/buffer.py
symbol_scope:
  - StreamHandler.consume
  - StreamHandler._flush
acceptance:
  - concurrent producers preserve emission order
  - sentinel terminates buffering atomically
  - downstream observers see one payload per
    logical message
parents: [unit_003, unit_005]
tests:
  - tests/unit_007/test_ordering.py
  - tests/unit_007/test_sentinel.py
  - tests/unit_007/test_concurrent.py
```

The requirement is phrased as an acceptance contract. Procedural directives, command level instructions, and references to the gold patch are omitted. The resulting loop trace records whether the evaluated loop infers ordering and integration constraints from the dependency graph and released tests without access to the hidden implementation path.

Recovered DAG. The recovered DAG encodes the routing and obligation structure recorded by the evaluation runtime. DAG recovery (Section 2.4) admits **18** edges over the 12 units under the unambiguous evidence rules. The longest dependency chain has depth **6**. The maximum out degree is **4**, corresponding to a foundational refactor unit imported by later units. Three units form an independent branch with zero incoming edges from the rest of the task, creating a parallel ready frontier midway through the PR sequence.

Trial verification. The materialized test suite passes the solvability, non-triviality, and discriminativeness trials of Section 2.5 on the first run for **10** of **12** units. Two units undergo one repair round during materialization. In one case a flaky timing assertion is replaced with a deterministic state check. In the other, a discriminativeness trial reveals that two predecessor tests already cover the unit’s full contract, and the materialization pipeline strengthens the unit suite with an additional assertion targeting the new contract.

Evaluation trace excerpt. Under the protocol of Section 2.6, the recorded loop trace contains **31** checkpoints over the evaluation episode. The visitation sequence σ differs from the human order σ^* on **3** of 12 units, all within branches with partially ordered alternatives under the DAG. Two regressions are recorded against previously completed units, and one is recovered before the episode ends. The full test state matrix S is omitted for brevity, while Appendix G defines the corresponding loop trace metrics.

E Contamination Checks for Loop Engineering Evaluation

LOOPSBENCH derives its loop engineering evaluation tasks from public development artifacts, including PR Sequences, Course Labs, and Research Evolutions. This source choice creates potential overlap with modern LLM pretraining data. We therefore separate source recognition risk from the loop trace metrics used to evaluate loop execution.

Input Side Anonymization. During Task Instrumentation (Section 2.5), every unit requirement is rewritten to remove identifiable tokens that could let a model recognize the source from the prompt: repository names, course identifiers (e.g., MIT 6.824, Stanford CS144), paper titles, and author attributions. For PR Sequences, the same rewriting policy is applied at the segment (SEG) level, where original commit messages and PR titles often include the feature name, issue tracker ID, or merging author. After this step, the task prompt provided to the evaluated loop contains the acceptance specification, public interfaces, and tests. Source identifying evidence that would support retrieval by name is removed before evaluation.

Memorization and Loop Trace Metrics. Anonymization addresses prompt level identification, but training set memorization may remain in model weights. A model that has internalized a reference implementation, for example, may reproduce local algorithmic fragments regardless of prompt wording. Two observations limit the extent to which this residual risk explains the loop trace metrics recorded by LOOPSBENCH.

The first observation is empirical. The strongest configuration in our evaluation, Claude Code with Opus-4.7, uses the native Anthropic model and loop infrastructure under the most permissive context budget and reaches a Resolve Rate of 25.00% (Table 2). If weight level memorization were the principal driver, frontier model and loop configurations would be expected to approach higher resolution on canonical Course Labs and Research Evolutions tasks, whose source material is publicly accessible. This result provides limited support for memorization as the main explanation.

The second observation is structural. LOOPSBENCH treats recall of a single implementation fragment as insufficient evidence for loop engineering evaluation. A resolved task entails prerequisite consistency across many edits, retained obligations as the codebase evolves, preserved state across context renewal, and test feedback routed along the dependency DAG. These loop conditions extend beyond

pretraining exposure to a solution for a single local unit. Memorization may supply fragments, but LOOPS BENCH records whether those fragments remain consistent under live regression obligations, where current model and loop configurations exhibit constrained state continuity and obligation retention.

Model cutoff robustness slice. A direct slice by source ingest date and model cutoff can partition tasks by whether their source material predates each model’s training cutoff and report Resolve Rate within each partition. The current analysis reports the aggregate constraint that the strongest model and loop configuration reaches 25% Resolve Rate, providing limited support for memorization driven inflation as the principal explanation.

F Evaluation Settings for Model and Loop Configurations

Models and provider routes. RQ1 (E1) isolates model choice by comparing eight model checkpoints under a fixed Claude Code loop implementation: Opus-4.7, GPT-5.5, GLM-5.1, DeepSeek-V4P, Gemini-3.1-Pro, Qwen3.6-Plus, Kimi-2.6, and Grok-4.1-FR. RQ1 (E2) isolates loop implementation by fixing the model to `azure/gpt-5.4-20260305` and varying Claude Code, Codex, GitHub Copilot, OpenHands, SWE-agent, and mini-SWE-agent. RQ1 (E3) examines within family model changes while pairing each family with its corresponding loop implementation for GPT, Claude Opus, and Qwen. Provider access is normalized through routes implemented in `long_horizon_bench/agents/`, including an Azure OpenAI deployment with AD token auth (`api-version 2025-04-01-preview`), the GitHub Copilot token exchange endpoint, a DashScope Anthropic compatible proxy, and direct Anthropic SDK access for Anthropic native models. This normalization separates provider transport from the loop engineering factors attributed to model choice and loop implementation.

Decoding and evaluation input. Each loop implementation inherits provider defaults unless its evaluation adapter records an explicit override. For the Anthropic SDK path used by Claude Code, we set `max_tokens=128,000` and leave temperature and top p at their SDK defaults. The SWE-agent LiteLLM call uses `temperature=0`. Top p is disabled because Anthropic rejects requests that set both fields. mini-SWE-agent forwards `model.model_kwargs.reasoning_effort=high` to providers that accept it. Loop implementations receive the evaluation contract in Appendix P; no additional custom system prompt is supplied unless the evaluation adapter defines one.

Runtime budget and concurrency. The evaluation runtime imposes a per task wall clock budget $T_{\max}$ chosen per task (≥ 2 h, capped at 24 h for the longest PR Sequence and Research Evolution tasks) and passes it to each loop implementation as `timeout_sec`. For loop implementations with their own iteration cap, we set the cap above the wall clock budget, making wall clock time the evaluation horizon. Claude Code SDK uses `max_iterations=10,000`, and mini-SWE-agent uses `-cost-limit 0` (uncapped) with `-exit-immediately`. Each bash tool call inside the container is independently capped at 600 s, separating local command stalls from sustained loop progress. Trials run concurrently in a `ThreadPoolExecutor`. Per trial credentials are passed as explicit kwargs through the task runner, keeping cross trial authorization races outside attribution to the evaluated loop implementation.

G Metrics for Loop Engineering Evaluation

We formalize the five headline RQ1 metrics used to evaluate model and loop configurations on LOOPS BENCH. Together, they separate end state scoring from loop trace metrics for dependency progress, obligation retention, and resource use. Let a task have tests indexed by i and evaluation checkpoints indexed by $\tau \in \{1, \dots, T\}$. Let $s_i^{(\tau)} \in \{0, 1\}$ denote the pass/fail outcome of test i at checkpoint τ , and let O_τ denote the set of *regression obligations* active at checkpoint τ (tests previously released and passed at some earlier checkpoint). The recorded test state matrix is $S = [s_i^{(\tau)}]$ and serves as the evidence basis from which both final outcomes and intermediate loop trace metrics are computed. Let D_τ denote the set of tests released by checkpoint τ , and let $T_{\max}$ denote the wall

clock time budget. The released test pass rate at checkpoint τ is

$$\text{TPR}(\tau) = \frac{|\{i \in D_\tau : s_i^{(\tau)} = 1\}|}{|D_\tau|}. \quad (1)$$

G.1 Resolve Rate (RR)

$$\text{RR} = \mathbb{I} \left[\bigwedge_{i \in \text{all tests}} s_i^{(T)} = 1 \right]. \quad (2)$$

RR records whether every released test passes at task end, including regression obligations accumulated as tests are released. It is the strict end state outcome metric, while the remaining metrics record dependency progress, obligation retention, and resource use along the recorded loop trace.

G.2 Test Pass Rate (TPR)

$$\text{TPR} = \frac{1}{|D_T|} \sum_{i \in D_T} s_i^{(T)}. \quad (3)$$

TPR records the average fraction of released tests passing at task end. Unlike RR, it preserves partial progress over the released test set and connects the final state to the intermediate loop trace recorded during execution.

G.3 Dependency Depth (Depth)

Depth measures dependency reach before the first residual handoff by the outer evaluation loop, isolating how far one continuous execution segment preserves prerequisite obligations before residual continuation begins. Let each released test i inherit the topological layer of the development unit it adjudicates, $\text{layer}(i) \in \{0, \dots, L\}$, where $L = \max_{v \in V} \text{layer}(v)$ is the longest prerequisite chain of the task DAG. Let $\tau^\dagger$ denote the last checkpoint of the first outer evaluation round, that is, the final snapshot before the outer loop, if any, restarts the loop with a freshly derived residual. Let $P_{\tau^\dagger} = \{i : s_i^{(\tau^\dagger)} = 1\}$ denote the tests passing at $\tau^\dagger$. Depth is the deepest layer at which every prerequisite test still passes at $\tau^\dagger$, normalized by L :

$$\text{Depth} = \mathbb{E}_{\text{task}} \left[\frac{\max\{\ell : \forall i, \text{layer}(i) \leq \ell \Rightarrow i \in P_{\tau^\dagger}\}}{L} \right]. \quad (4)$$

Depth records dependency reach before the first residual handoff in the evaluated loop, while RR combines this reach with residual recovery across later rounds. Resolved tasks may reach $\text{RR} = 1$ through high Depth before the first residual handoff or through lower initial Depth followed by successful residual recovery. Read together, the two metrics separate entry layer stalls ($\text{Depth} \ll 1$) from late prerequisite discontinuities, where high Depth pairs with low RR after later execution no longer preserves prior obligations.

G.4 Regression Rate (Reg)

A *regression event* occurs whenever a previously passing obligation test fails at a later evaluation point, making obligation retention observable as a loop trace metric rather than solely as final nonresolution. Checkpoints τ are snapshots emitted by the dual container watcher (Section 2.6). When the cumulative diff from the evaluated loop exceeds a fixed line threshold against the snapshot baseline, the watcher container freezes the workspace and runs the full released test suite, recording per test pass or fail outcomes in `observation_history.jsonl` alongside the `newly_passed` and `newly_failed` deltas relative to the previous snapshot. Let τ^- denote the immediately preceding triggered snapshot at which test i was evaluated, let O_τ denote the obligation set at τ (the union of all tests that were `newly_passed` at any earlier triggered snapshot), and define

$$R = \#\{(i, \tau) : i \in O_\tau, s_i^{(\tau^-)} = 1, s_i^{(\tau)} = 0\}. \quad (5)$$

Then

$$\text{Reg} = \frac{R}{\sum_{\tau} |O_{\tau}|}. \quad (6)$$

Equivalently, Reg measures how often a previously satisfied obligation stops passing within the recorded loop trace, making obligation retention a directly measured loop trace metric. Reg is interpreted jointly with Resolve Rate. Configurations with very low Resolve Rate satisfy a limited obligation set, and $|O_{\tau}|$ therefore provides limited exposure for estimating retention. Low Reg in that regime provides limited evidence of stable long horizon execution. Conversely, a model and loop configuration with moderate RR but high Reg, such as DeepSeek-V4P at RR=11.61% and Reg=16.60% in our runs, corresponds to obligation retention degradation beyond limited progress alone. We report Reg as a loop trace metric and use Resolve Rate for outcome ranking.

G.5 Tokens (Tok)

Tok records total tokens consumed per task, summed across input and output over the full loop trace. This metric reports the resource cost of sustaining the evaluated loop over long horizon execution and contextualizes loop trace improvements against runtime expenditure.

H Model Metadata for Fixed Loop Comparisons

For the within family scaling panel in Table 2, we record release date and reported parameter scale for each model evaluated under a fixed loop implementation. The three families begin at different release points and follow different release cadences. The GPT family spans gpt-4o (May 2024) through gpt-5.5 within the RQ1 evaluation window, the Claude family spans Sonnet-4 through Opus-4.7, and the Qwen family spans Qwen2.5-72B (open weight, 72 B parameters) through Qwen3.6-Plus (closed weight, undisclosed scale). Because the columns mix open and closed weight checkpoints, the metadata contextualizes fixed loop comparisons rather than defining a model capability ranking. Resolve Rate monotonicity within each family is therefore interpreted as a generational delta under a fixed loop implementation, not as a parameter count scaling law.

Table 9: Model metadata for RQ1 comparisons.

Model	Family	Release	Params	Ref.
GPT-4o	GPT	2024-05	undisclosed	(OpenAI, 2024a)
GPT-5	GPT	2025-08	undisclosed	(OpenAI, 2025b)
GPT-5.2	GPT	2025-11	undisclosed	(OpenAI, 2025c)
GPT-5.4	GPT	2026-03	undisclosed	(OpenAI, 2026b)
GPT-5.5	GPT	2026-04	undisclosed	(OpenAI, 2026c)
Sonnet-4	Claude	2025-05	undisclosed	(Anthropic, 2025a)
Sonnet-4.5	Claude	2025-09	undisclosed	(Anthropic, 2025c)
Opus-4.5	Claude	2025-11	undisclosed	(Anthropic, 2025b)
Opus-4.7	Claude	2026-02	undisclosed	(Anthropic, 2026d)
Qwen2.5-72B	Qwen	2024-09	72 B (OW)	(Qwen Team, 2024)
Qwen3-Max	Qwen	2025-09	undisclosed	(Alibaba Cloud, 2025)
Qwen3.5-Plus	Qwen	2025-12	undisclosed	(Alibaba Group, 2026)
Qwen3.6-Plus	Qwen	2026-03	undisclosed	(Qwen Team, 2026)
Gemini-3.1-Pro	Gemini	2026-03	undisclosed	(Google DeepMind, 2026)
GLM-5.1	GLM	2026-02	undisclosed	(Z.AI, 2026)
DeepSeek-V4P	DeepSeek	2026-01	undisclosed	(DeepSeek-AI, 2026)
Kimi-2.6	Moonshot	2026-03	undisclosed	(Moonshot AI, 2026)
Grok-4.1-FR	xAI	2026-02	undisclosed	(xAI, 2026)

I Pass@K Robustness Under Fixed Loop Implementations

The monotonic Resolve Rate trend across model generations within each family, reported in the left panel of Table 2, is estimated from one trial per model and loop pairing. We therefore also report pass@3 on the same model and loop pairings. The within family ordering is preserved at $K>1$ for every family, which supports the headline trend in Table 2 beyond single trial variance. The limited pass@3 rate is consistent with repeated trials under a fixed model and loop configuration rarely

overcoming the state continuity and regression obligation retention limitations observed during long horizon execution.

Table 10: Pass@K Resolve Rate for within family scaling.

Model	pass@1	pass@3
<i>GPT (Codex)</i>		
GPT-5.5	21.43	23.21
GPT-5.2	13.39	15.18
GPT-5	8.04	9.82
GPT-4o	3.57	4.46
<i>Claude (Claude Code)</i>		
Opus-4.7	25.00	26.79
Opus-4.5	17.86	19.64
Sonnet-4.5	12.50	14.29
Sonnet-4	7.14	8.93
<i>Qwen (Qwen Code)</i>		
Qwen3.6-Plus	9.82	11.61
Qwen3.5-Plus	8.04	9.82
Qwen3-Max	1.79	2.68
Qwen2.5-72B	0.00	0.89

J Per Task Billed Tokens for RQ1

Tok reports mean billed tokens per task in millions under outer continuation, matched to the RQ1 configurations in Table 2.

K Planning Loop Trace Metrics for Loop Engineering Evaluation

We formalize the six planning metrics used in RQ2 for loop engineering evaluation. They compare the plan DAG recorded in the evaluated loop trace with the source recovered human DAG and summarize how planning choices change routing burden and obligation retention pressure (Table 3). Let $G_A = (V_A, E_A)$ denote the recorded plan DAG and $G_H = (V_H, E_H)$ the human development DAG recovered from source records. Let $V_\cap = V_A \cap V_H$ denote the set of units present in both DAGs.

K.1 Edge F1

Let $E_\cap = E_A \cap E_H$, and define

$$P_{\text{edge}} = \frac{|E_\cap|}{|E_A|}, \quad R_{\text{edge}} = \frac{|E_\cap|}{|E_H|}. \quad (7)$$

$$\text{Edge F1} = \frac{2 P_{\text{edge}} R_{\text{edge}}}{P_{\text{edge}} + R_{\text{edge}}}. \quad (8)$$

Edge F1 measures exact recovery of direct prerequisite edges. Low precision corresponds to unsupported dependencies in the recorded plan, while low recall corresponds to omitted prerequisites that can cause work to be routed before prerequisite state is established.

K.2 TC-F1

Let $\text{TC}(\cdot)$ denote the transitive closure of a DAG’s edge set, and write $T_\cap = \text{TC}(E_A) \cap \text{TC}(E_H)$. Define

$$P_{\text{tc}} = \frac{|T_\cap|}{|\text{TC}(E_A)|}, \quad R_{\text{tc}} = \frac{|T_\cap|}{|\text{TC}(E_H)|}. \quad (9)$$

$$\text{TC F1} = \frac{2 P_{\text{tc}} R_{\text{tc}}}{P_{\text{tc}} + R_{\text{tc}}}. \quad (10)$$

Table 11: Mean billed tokens per task (millions) for the RQ1 configurations.

Configuration	Tok↓
<i>GPT (Codex)</i>	
GPT-5.5	8.36M
GPT-5.2	9.10M
GPT-5	8.92M
GPT-4o	4.51M
<i>Claude (Claude Code)</i>	
Opus-4.7	6.91M
Opus-4.5	5.38M
Sonnet-4.5	4.21M
Sonnet-4	3.76M
<i>Qwen (Qwen Code)</i>	
Qwen3.6-Plus	6.96M
Qwen3.5-Plus	6.88M
Qwen3-Max	5.71M
Qwen2.5-72B	5.02M
Model (fixed loop: Claude Code)	
Opus-4.7	6.91M
GPT-5.5	7.18M
GLM-5.1	4.37M
DeepSeek-V4P	3.59M
Gemini-3.1-Pro	4.99M
Qwen3.6-Plus	4.25M
Kimi-2.6	2.02M
Grok-4.1-FR	1.83M
Loop (fixed model: gpt-5.4)	
Codex	7.94M
Claude Code	7.03M
GitHub Copilot	8.26M
OpenHands	6.75M
SWE-agent	7.18M
mini-swe-agent	8.86M

TC-F1 is a more permissive measure of prerequisite ordering agreement. It credits a recorded plan that preserves u *before* v even when it uses a different intermediate path from the human development DAG, capturing whether broad routing order survives edge level mismatch.

K.3 Layer ρ

Let $\ell_A(v)$ and $\ell_H(v)$ denote the topological layer index of unit v in G_A and G_H . Layer 0 contains prerequisite free nodes, and later layers depend on earlier layers.

$$\text{Layer } \rho = \text{Spearman}(\{\ell_A(v)\}_{v \in V_\cap}, \{\ell_H(v)\}_{v \in V_\cap}). \quad (11)$$

Layer ρ records whether the plan places units at similar development *stages* as the human reference, independent of exact edge structure. Low values correspond to the evaluated loop assigning units to different prerequisite layers, which changes when obligations become ready during execution.

K.4 Critical Path Ratio

Let $\text{CP}(G)$ denote the length (in nodes) of the longest directed path in G .

$$\text{CPR}(G) = \frac{\text{CP}(G)}{|V|}. \quad (12)$$

Values near 1 correspond to a nearly linear recorded plan that overserializes the task. Lower values correspond to preservation of parallel branches that can be routed independently. We also report a

human normalized variant $\text{CPR}(G_A)/\text{CPR}(G_H)$ in extended results to separate task shape from loop planning behavior.

K.5 Width Ratio

Let $w(G) = \max_{\ell} |\{v \in V : \ell(v) = \ell\}|$ denote the maximum topological layer width of G .

$$\text{WR} = \frac{w(G_A)}{w(G_H)}. \quad (13)$$

Values close to 1 correspond to a recorded plan that matches the human development DAG’s concurrent ready frontier width. Values below 1 correspond to overserialization, while values above 1 are consistent with overparallelization caused by missing prerequisite edges and therefore higher regression obligation pressure.

K.6 Branch Switching Rate

Assign each unit v a branch identifier $b(v)$ corresponding to its earliest reachable source node in G_H (units with multiple source ancestors are labeled *mixed*). Given the planned or executed implementation order in the recorded loop trace $\sigma = (v_{\pi(1)}, \dots, v_{\pi(m)})$, let $K = |\{i : b(v_{\pi(i)}) \neq b(v_{\pi(i+1)})\}|$.

$$\text{BSR} = \frac{K}{m-1}. \quad (14)$$

High values correspond to breadth first frontier coverage across branches, while low values correspond to depth first completion of one branch before switching. BSR is graph conditional and is read relative to the shape of G_H , since the same switching pattern can reflect frontier coverage in a wide DAG or unnecessary alternation in a narrow chain.

L Extended RQ2 Loop Trace Metrics

Loop trace metric definitions for Table 3. *Planning fidelity.* **Edge F1** = F1 between the plan DAG edge set in the recorded loop trace and the human development DAG edge set. **Layer ρ** = Spearman correlation between topological layer indices in the recorded loop trace and those in the human development DAG. **CPR** = critical path length of G_A divided by $|V_A|$ (direct value, unnormalized to gold). **WR** = mean ready frontier width of G_A divided by that of G_H . *Implementation* metrics are computed on units resolved by the evaluated loop. **PatchLen** = patch lines per matched unit divided by gold patch lines. **Jacc** = token level Jaccard distance between candidate patches in the recorded loop trace and gold patch. *Testing* metrics summarize verification work authored during the recorded loop trace. **#T** = number of tests authored in the recorded loop trace per task. **F2P** = fraction of tests authored in the recorded loop trace that fail on the initial state and pass on the gold solution.

Auxiliary planning metrics. Table 3 presents a compact metric set for planning, implementation, and testing. The auxiliary metrics here either duplicate headline columns or depend more directly on task graph shape than on loop implementation. Their appendix placement keeps the main table focused on loop trace metrics that remain comparable across loop implementations.

TC-F1 (planning). Transitive closure F1 (Appendix K.2) is strongly correlated with Edge F1 (Appendix K.1) in our runs ($\rho > 0.9$ across the six evaluated loops), so it provides secondary evidence beyond the Edge F1 column in the main table. Its absolute level estimates whether the recorded plan captures *indirect* prerequisite structure relevant to routing and obligation retention.

Branch Switching Rate (planning). BSR (Appendix K.6) is graph conditional. Higher BSR is consistent with frontier exploration when G_H has many simultaneously ready branches, but on long sequential chains it can instead record unnecessary alternation across branches. We therefore report it as an extended loop trace metric outside the main table ranking columns.

Test-Ratio (testing). The main table reports #T as an absolute test count. Test-Ratio equals $\#T/\#T_{\text{native}}$ with $\#T_{\text{native}} = 44$, making it directly derivable from the #T column and secondary to the reported verification effort metric.

JSD (testing). JSD measures distributional divergence between tests authored during the recorded loop trace and native test placement. Appendix placement reflects its auxiliary role relative to the main #T and F2P columns, which more directly measure verification effort and fail to pass yield during loop engineering evaluation. Unit position histograms separate early unit concentration from late unit concentration, capturing where verification effort is routed along the horizon.

Source of additional patch lines. The main text reports that PatchLen exceeds the gold reference while Jacc to gold stays at a comparable moderate level. The recorded diffs explain the two metrics jointly. Candidate patches often impose a uniform style, add extensive comments, and rewrite existing repository blocks when repository state in the loop is incomplete. This pattern increases patch length while token level overlap remains close to the human reference, suggesting that loop state gaps can appear as implementation surplus rather than disjoint implementations.

M Per Source Resolve Rate for Model and Loop Configurations

The main experiments report aggregate Resolve Rate and loop trace metrics over the 112 LOOPS-BENCH tasks. This appendix reports the base RQ1 setting before outer continuation, matching the *w/o* RR column in Table 2. The three task sources represent distinct evidence regimes with different routing burden, state continuity, and regression obligation retention profiles. PR Sequences inherit source evidenced dependency DAGs from merged commit history, Course Labs inherit module level dependency structure from curriculum design, and Research Evolutions inherit coarser successor chains from citation graphs. Per source reporting separates these loop engineering patterns before aggregation. Table 12 reports per source Resolve Rate for every model and loop configuration evaluated in RQ1. This split places the aggregate comparison in source context and relates each source to the routing and obligation retention burdens faced by each loop implementation. The aggregate column reproduces the corresponding *w/o* number from Table 2 and equals $RR_{\text{agg}} = \frac{1}{112} (29 \cdot RR_{\text{PR}} + 57 \cdot RR_{\text{Course}} + 26 \cdot RR_{\text{Research}})$ up to rounding. Two findings hold uniformly across model and loop configurations and frame the interpretation of the per source columns.

Finding 1: PR Sequences are uniformly the most difficult source. Across all 14 model and loop configurations, PR Sequence Resolve Rate reaches at most 3.45%, and 9 of 14 configurations resolve no PR Sequence tasks. The PR Sequence pool is dominated by merge chains from actively maintained repositories, where each segment often depends on cross module invariants established several PRs earlier. When an early segment fails to preserve a prerequisite invariant, downstream progress becomes governed by obligation retention across dependent edits. This pattern is consistent with the RQ3 observation that regression events remain present across loop profiles, including runs with no recorded internal compaction. **Finding 2: Course Labs account for most resolved tasks.** For every model and loop configuration, Course Labs account for $\geq 70\%$ of the resolved task count. Course Labs have shallower dependency depth on average (median depth 3 vs. 6 for the pool) and a curriculum imposed module decomposition that often makes the human development order recoverable from task structure. Per source Resolve Rate on Course Labs varies most with model and loop configuration differences, and it tracks the aggregate Resolve Rate ranking most closely. Research Evolutions fall between the two extremes. The moderate pool size (26 tasks) provides a more reliable per source estimate than earlier preview cuts, while the strongest model and loop configurations resolve a limited number of tasks and the weakest configurations resolve none. At this sample size, the per source spread is read together with the column level rate.

These patterns shape the interpretation of aggregate LOOPS-BENCH Resolve Rate. A single aggregate comparison combines Course Labs variation with the limited spread in PR Sequences, and the loop sweep gap in Table 2 is driven largely by Course Labs. PR Sequence gains are clearer in the per source columns than in aggregate deltas. Doubling PR Sequence RR moves the aggregate by less than one percentage point while materially changing the loop engineering interpretation of routing burden and obligation retention.

Table 12: Resolve Rate by task source before outer continuation.

Loop	Model	PR Seq. (n=29)	Course (n=57)	Research (n=26)	All (n=112)
<i>Model sweep under the Claude Code loop</i>					
Claude Code	Opus-4.7	3.45%	24.56%	15.38%	16.96%
Claude Code	GPT-5.5	0.00%	21.05%	11.54%	13.39%
Claude Code	GLM-5.1	0.00%	21.05%	11.54%	13.39%
Claude Code	DeepSeek-V4P	0.00%	19.30%	7.69%	11.61%
Claude Code	Gemini-3.1-Pro	0.00%	14.04%	7.69%	8.93%
Claude Code	Qwen3.6-Plus	0.00%	10.53%	3.85%	6.25%
Claude Code	Kimi-2.6	0.00%	5.26%	3.85%	3.57%
Claude Code	Grok-4.1-FR	0.00%	5.26%	0.00%	2.68%
<i>Loop sweep under a fixed gpt-5.4 model</i>					
Codex	gpt-5.4	0.00%	21.05%	11.54%	13.39%
Claude Code	gpt-5.4	0.00%	19.30%	11.54%	12.50%
GitHub Copilot	gpt-5.4	0.00%	17.54%	7.69%	10.71%
OpenHands	gpt-5.4	0.00%	10.53%	3.85%	6.25%
SWE-agent	gpt-5.4	0.00%	8.77%	3.85%	5.36%
mini-swe-agent	gpt-5.4	0.00%	8.77%	0.00%	4.46%

N Context Round and Regression Metrics

N.1 Trace Definitions

RQ3 counts both external restarts and compaction intervals as context-budget rounds. For trial i , let K_i be the number of outer invocations and let C_i be the number of context segments implied by the implementation compaction policy. Each segment opens a new maximal context budget. We report

$$\text{Rounds} = \frac{1}{N_{\text{trace}}} \sum_{i=1}^{N_{\text{trace}}} (K_i + C_i). \quad (15)$$

An external restart replaces the active context and is counted in K_i . An internal compaction interval is counted in C_i . Codex uses the source-level auto-compaction limit, which defaults to 0.9 of the configured context window. Claude Code uses its configured context budget, and the Ralph SDK analysis uses the SDK compaction interval. Dynamic workflows sum context-budget rounds across the coordinating workflow and worker contexts.

The regression watcher writes `observation_history.jsonl`. Let $F_{i,s}$ be the set of released tests that newly fail at triggered snapshot s . Regression events per run are

$$\text{Reg/run} = \frac{1}{N_{\text{trace}}} \sum_{i=1}^{N_{\text{trace}}} \sum_s \mathbb{I}[F_{i,s} \neq \emptyset]. \quad (16)$$

Resolve Rate uses the benchmark outcome for every scheduled task:

$$\text{RR} = \frac{1}{N_{\text{task}}} \sum_{i=1}^{N_{\text{task}}} \mathbb{I}[\text{trial } i \text{ is resolved}]. \quad (17)$$

N.2 Run Coverage

The Codex goal, Claude Code goal, and Ralph runs each contain 102 scheduled tasks. The dynamic workflow run contains 112 scheduled tasks. The corresponding run identifiers are `stage4-C1-codex-gpt55-0858`, `cc-cli-opus46-goal-1714`, `stage4-A1-opus47-1442`, and `cc-dynamic-opus48-20260705-043454`. Table 4 uses the available run summaries for Rounds, the available regression histories for Reg/run, and all scheduled tasks for RR.

N.3 Trace Scope

The dynamic workflow archive records the coordinating Claude Code session and outer restarts. It does not retain a uniform event stream for every worker process. Its Rounds value therefore describes the top level workflow context. Worker completion state remains visible through repository changes, requirement patches, and regression snapshots.

The four runs use their native model and loop configurations. Their values characterize observed loop profiles and are not a model controlled causal comparison. Missing outer histories are excluded from Rounds and Reg/run, while all scheduled tasks remain in the RR denominator. The figure script reads the archived JSON files directly and applies these rules without manual value entry.

O Evaluation Adapter Details

LOOPSBENCH evaluates six widely used coding agent loop implementations through a uniform BaseAgent contract. The evaluation adapters normalize credential handling, model identifiers, and workspace conventions, so Table 2 can vary model choice and loop implementation while holding the task contract fixed.

Each run records the evaluation adapter boundary because it determines how an evaluated loop receives requirements, commits work, records plans, and returns artifacts used to compute loop trace metrics.

Workspace and git contract. Each evaluated loop executes inside /workspace in a per task Docker container built from the task’s Dockerfile, providing every adapter with the same filesystem and execution boundary.

The directory is initialized as a git repository with three subtrees recorded by the evaluation runtime. requirements/ contains per unit YAML manifests presented during evaluation, agent_plans/ stores the optional plan.json for RQ2 plan analysis, and requirement_patches/ stores one <slug>.diff per implemented unit derived from git diff -cached.

A unit enters the recorded implementation set after its patch file is non empty. Checkpoint detection (long_horizon_bench/harness/regression_harness.py) uses min_diff_lines=5 as the minimum diff growth for a snapshot trigger. The per unit threshold is derived from the smallest gold artifact currently on the ready frontier, making snapshot frequency follow recorded implementation progress rather than transcript length or turn count.

Evaluation adapters. Claude Code is evaluated through the Anthropic Agent SDK with a ContainerWorkspaceBackend that proxies the SDK bash and edit tools into the task container. Loop traces and per trial usage are written to claude_sdk_trajectory.jsonl and claude_sdk_result.json.

Codex uses a custom model_provider block, lhb_azure or lhb_proxy, injected into config.toml. This preserves the Codex invocation contract while routing model calls through the evaluation provider layer, keeping provider transport separate from loop behavior.

GitHub Copilot CLI is evaluated in noninteractive mode against a sidecar at AZURE_SIDECAR_PORT=23337, reached through the container default gateway IP in default compose setups that omit host.docker.internal.

Cursor CLI uses an adapter that maps Anthropic shorthand to Cursor model IDs before invoking agent -model.

mini-SWE-agent is installed inside the container at /opt/python through pip -prefix with an explicit PYTHONPATH adjustment. Models prefixed with copilot/ are rewritten to openai/, allowing LiteLLM to reach the OpenAI compatible proxy.

SWE-agent applies a compatibility patch that removes the top_p keyword. Anthropic rejects requests carrying both temperature and top_p, while the SWE-agent defaults set both fields.

Credential isolation. Earlier versions of the evaluation adapter wrote Azure AD tokens and Copilot keys into os.environ inside a context manager. Under the ThreadPoolExecutor used by the

evaluation runtime, concurrent trials could overwrite shared tokens, and cleanup from one trial could clear a credential entry while another trial remained active. The current evaluation adapters pass credentials as explicit kwargs to `run_task` (`azure_ad_token_provider`, `copilot_api_key`, `anthropic_api_key`), preventing credential races from being attributed to the evaluated loop.

P Evaluation Contract for Loop Engineering

The evaluation contract supplied to each loop implementation is composed by `long_horizon_bench/harness/instruction_composer.py`. It combines task wording, planning requirements, and evaluation adapter addenda into the reproducible input observed by the evaluated loop.

For each evaluated loop, the input combines the task instruction, the *Benchmark Contract* (always), the *DAG Plan Requirement* during RQ2 plan fidelity runs, and a profile addendum selected by evaluation adapter class. The Oracle adapter bypasses composition.

The literal prompt text below remains unchanged because it is the evaluated input. The surrounding prose records how that input defines the loop engineering evaluation contract.

Benchmark Contract (always prepended; `instruction_composer._BENCHMARK_CONTRACT`)

You are running as a long-horizon autonomous engineering agent at ``/workspace``. This task spans many files and requirements, and is expected to take many iterations.

Read every requirement under ``/workspace/requirements/`` (and any other explicit requirement list referenced by the task) and only finish once every single one is addressed. Cherry-picking or partial fixes will lead to failure.

Optional planning artifact

If you believe the task is complex enough to benefit from a plan, you may write ``/workspace/agent_plans/plan.json`` before making changes. This is optional. If you create it, it must be a JSON object with ``nodes`` and ``edges``. Each node must have string ``id`` equal to a requirement slug from ``/workspace/requirements/``, may include ``label`` and ``layer``, and each edge must use string ``from`` and ``to`` that reference those ids.

Git workflow (required)

``/workspace`` is initialized as a git repo. After finishing the implementation for ONE requirement:

1. ``cd /workspace``
2. ``git add -A``
3. ``git diff --cached > requirement_patches/<slug>.diff``
4. ``git commit -m "impl: <slug>"``

Where ``<slug>`` is the requirement filename without the ``.yaml`` extension. The patch file MUST be non-empty for the requirement to count as complete. Never write or ``touch`` an empty ``.diff`` file.

You may run ``git log``, ``git diff``, ``git show`` at any time to inspect what prior work has done -- only your own commits will be there; no reference solution is accessible.

Recommended workflow

1. Read the task carefully and survey ``/workspace`` so you understand the code you are changing.
2. As a development agent, when implementing a requirement you may write tests under ``/workspace/agent_tests/`` to safeguard correctness and future stability -- write them if you think they help, otherwise skip.
3. Keep iterating until every requirement is implemented. Do not stop at a partial fix, a first passing check, or one successful edit.
4. Do not narrow the scope on your own. Missing a requirement is a task failure even if the changed tests pass.
5. You may revise the same files and rerun focused checks as many times as needed before finishing.
6. Your final summary must report requirement coverage counts, including at least the total number of requirements reviewed and how many were implemented and blocked.

Required DAG Plan (replaces the optional planning paragraph for RQ2 plan-fidelity runs)

Required planning artifact (DAG)

Before making any code changes you MUST write ``/workspace/agent_plans/plan.json``. It must be a JSON object with ``nodes`` and ``edges`` representing a directed acyclic graph (DAG) over the requirement set. The DAG should reflect your view of which requirements can be implemented in parallel and which must be serialized:

- ``nodes``: an array. Each node is `{ "id": "<slug>", "label": "...", "layer": <int> }`. ``id`` MUST exactly equal a requirement filename under ``/workspace/requirements/`` without the ``.yaml`` extension. Every requirement slug MUST appear as a node -- no omissions.
- ``edges``: an array. Each edge is `{ "from": "<slug>", "to": "<slug>" }` meaning the ``from`` requirement must be implemented before ``to`` (a real ordering constraint, e.g. shared file/function, API surface change, or a logical prerequisite). The graph MUST be acyclic. Both endpoints MUST exist in ``nodes``.
- ``layer`` is your topological-layer index (0 for sources). Two nodes in the same layer should be safely parallelizable -- no edge between them and no implicit conflict.

Implement requirements in an order consistent with the DAG. If you discover the dependency structure is wrong while implementing, update ``plan.json`` first, then continue. Do NOT skip writing this file.

Profile – strong (Claude Code)

Agent profile:

- Focus on completing the task directly.
- Do not spend tokens on basic shell tutorials or extended environment narration.
- Do not silently narrow the task scope, claim success early, or stop after partial requirement coverage.
- Use concise progress updates, keep modifying the workspace when needed, and validate the parts you change.

Profile – structured_open_source (SWE-agent, OpenHands)

Agent profile:

- Work step by step and keep actions grounded in the repository state.
- Prefer short, concrete progress updates and targeted validation.
- Avoid unnecessary tutorial-style shell explanation.

Profile – legacy_verbose (mini-SWE-agent)

Agent profile:

- You may use a slightly more explicit step-by-step style when it helps execution.
- Keep progress updates concrete and repository-specific.
- Prefer direct validation of changed behavior before finishing.

For Codex, GitHub Copilot, and Cursor, the evaluation adapter leaves the profile addendum empty, so the evaluated loop receives the Benchmark Contract and, when applicable, the DAG plan replacement.

Q Loop Engineering Evaluation Runtime Configuration

The LOOPSBENCH evaluation runtime is invoked via `lhb run -config config.yaml`, and the resulting run artifact records the loop parameters used for attribution. The evaluation runtime treats the evaluation adapter, loop budget, and residual handoff policy as explicit loop engineering factors for each run. All parameters can be set in the YAML configuration file or overridden by CLI flags. Table 13 summarizes the runtime parameters used to separate attribution across model choice, evaluation adapter, budget, and residual handoff policy while keeping the evaluated task contract fixed.

Table 13: Runtime parameters for `lhb run`.

Group	Parameter	Default	Description
Dataset	<code>dataset_path</code>	<code>tasks/</code>	Path to the task directory tree.
	<code>task_ids</code>	<code>all</code>	Task ID globs to include (repeatable).
	<code>exclude_task_ids</code>	<code>[]</code>	Task ID globs to exclude (repeatable).
Evaluation adapter	<code>agent</code>	<code>oracle</code>	Built in evaluation adapter name.
	<code>agent_import_path</code>	<code>-</code>	Import path for a custom evaluation adapter class.
	<code>model</code>	<code>-</code>	LLM in <code>provider/model</code> format.
	<code>agent_kwargs</code>	<code>{}</code>	Key value pairs forwarded to the evaluation adapter.
Output	<code>output_path</code>	<code>runs/</code>	Root directory for run artifacts.
	<code>run_id</code>	<code>timestamp</code>	Unique identifier for this run.
	<code>n_tasks</code>	<code>all</code>	Cap on number of tasks to evaluate.
Build	<code>no_rebuild</code>	<code>false</code>	Skip rebuilding Docker images.
	<code>cleanup</code>	<code>true</code>	Remove images after run completes.
Logging	<code>log_level</code>	<code>info</code>	Python logging level.
	<code>livestream</code>	<code>false</code>	Stream evaluated loop I/O in real time.
Concurrency	<code>n_concurrent</code>	<code>4</code>	Tasks evaluated in parallel.
	<code>n_attempts</code>	<code>1</code>	Attempts per task.
Timeout	<code>global_timeout_multiplier</code>	<code>1.0</code>	Multiplier for all per task timeouts.
	<code>global_agent_timeout_sec</code>	<code>per task</code>	Override loop budget (seconds).
	<code>global_test_timeout_sec</code>	<code>per task</code>	Override test budget in seconds.

Per task timeouts. Each task declares `max_agent_timeout_sec` (default 1800s) and `max_test_timeout_sec` (default 3600s) in its `task.yaml`. The global overrides and multiplier allow uniform budget scaling without rewriting individual task files. Budget changes therefore remain attributable to run configuration rather than task materialization.

Residual handoff structure. The evaluation runtime executes an *outer loop* over evaluation adapter runs. Each round starts a fresh shell session while preserving the workspace from prior rounds, so residual handoff is measured as an observed loop engineering factor rather than treated as an implicit retry mechanism. The number of rounds is controlled by `outer_loop_count` (default 3); setting it to 0 or unlimited removes the cap, allowing repeated residual handoffs until the timeout budget is exhausted and making continuation policy explicit in the loop trace. Within each round, rate limit errors trigger retries governed by `rate_limit_retries_per_round` (default 5) with exponential backoff starting at `rate_limit_backoff_sec` (default 45s), recording provider availability separately from loop progress.

Model and provider configuration. Provider credentials and environment variables are supplied via a separate YAML file (`model_config_path` or `--model-config`), which supports `env_files` (dotenv paths) and an explicit `env` map. This separation places provider transport outside the loop engineering factors attributed to model choice and evaluation adapter. Keys already set in the process environment are preserved unless `override_process_env: true` is selected, reducing cross run configuration drift.

Example configuration. The following minimal configuration specifies the dataset and execution policy while recording the evaluation adapter, loop budget, and residual handoff policy as explicit run metadata.

Run Configuration (`lhb.run.yaml`)

```
dataset_path: tasks
output_path: runs

agent: mini-swe-agent
```

```
model: anthropic/claude-sonnet-4-20250514
```

```
n_concurrent: 4
```

```
n_attempts: 1
```

```
no_rebuild: false
```

```
cleanup: true
```

```
log_level: info
```

```
livestream: false
```

```
agent_kwargs:
```

```
  outer_loop_count: 3
```

```
  rate_limit_retries_per_round: 5
```

```
  rate_limit_backoff_sec: 45
```

```
global_timeout_multiplier: 1.0
```

Model Environment (lhb.model.yaml)

```
env_files: []
```

```
override_process_env: false
```

```
env:
```

```
  ANTHROPIC_API_KEY: "sk-ant-..."
```